



OPERATING SYSTEM



Third Level Students

Dr.Amira Eleff

Contents

Contents	2
1 Computer System Architecture	6
1.1 Basic Elements	7
1.2 Microprocessors: The Foundation of Modern Computing	10
1.3 GPUs: High-Performance Parallel Computing	11
1.4 Instruction Executions:	13
1.5 The Memory Hierarchy	29
1.6 Cache Memory	33
1.7 Direct Memory Access	37
1.8 Multiprocessor and Multicore Organization	38
1.9 Exercises	41
2 Operating System Overview	50
2.1 Operating System Objectives and Functions	50
2.2 Instruction Set Architecture (ISA)	52
2.3 Application Binary Interface (ABI)	53
2.4 Application Programming Interface (API)	53
2.5 The Evolution of Operating Systems	56

2.6	Major Achievements	63
2.7	Developments Leading to Modern Operating Systems .	68
2.8	Exercises	75
3	Process Description and Control	83
3.1	What Is a Process?	83
3.2	Process States and life Cycle	85
3.3	Process Description	110
3.4	Process Control	113
3.5	Exercise	120
4	Threads	126
4.1	Introduction and Motivation	126
4.2	Multicore Programming	127
4.3	Multi threading Models	128
4.4	Thread Libraries	130
4.5	Implicit Threading	131
4.6	Threading Issues	131
4.7	Operating System Examples	132
4.8	Exercises	133
5	Memory Management	137
5.1	Memory Management Requirements	137
5.2	Memory Partitioning	143
5.3	Dynamic Partitioning	148
5.4	Non-contiguous memory allocation(paging , Segmentation)	153
5.5	Exercise	171

6	Uni Processor Scheduling	174
6.1	Introduction to Scheduling in Operating Systems . . .	174
6.2	types of scheduling	176
6.3	Understanding the CPU-I/O Cycle	179
6.4	Scheduling Criteria	180
6.5	Scheduling Algorithms	183
6.6	Exercise	195
6.7	Solve the Problems	199
	Bibliography	201

Introduction

In the digital age, operating systems form the invisible backbone of modern computing. Whether powering smartphones, laptops, servers, or embedded systems, the operating system (OS) is the essential software layer that enables hardware to serve human needs. It manages resources, coordinates processes, enforces security, and provides a user interface—all while maintaining efficiency and stability in increasingly complex environments.

This ebook is designed to guide readers through the principles, design strategies, and practical implementations of operating systems. From fundamental concepts such as process scheduling and memory management to advanced topics like virtualization and distributed systems, each chapter builds a structured understanding of how operating systems work and why they matter.

Chapter 1

Computer System Architecture

An **operating system (OS)** serves as a vital intermediary between computer hardware and the users or applications that depend on it. Fundamentally, the OS utilizes the computational capabilities of one or more *central processing units (CPUs)* to coordinate and manage system resources, thereby delivering a suite of essential services that facilitate user interaction and application execution.

In addition to managing processor resources, the OS is responsible for controlling the system's *secondary memory*, such as hard disk drives (HDDs) and solid-state drives (SSDs). These storage devices retain data and programs even when the system is powered off. The OS ensures efficient and reliable data access, handles memory allocation, organizes files, and enforces access permissions to maintain system integrity and performance.

Furthermore, the OS oversees the operation of *input/output (I/O) devices*, which include peripherals like keyboards, mice, printers, monitors, and network adapters. It abstracts the complexity of hardware-specific operations by providing standardized interfaces that allow applications to interact seamlessly with these devices. This abstraction simplifies

development and enhances compatibility across different hardware platforms.

Given the OS's deep integration with the physical components of a computer system, it is essential to have a foundational understanding of the underlying hardware architecture. This includes knowledge of processors, memory hierarchies, buses, and device controllers. Such understanding provides valuable context for exploring the design, functionality, and evolution of modern operating systems.

1.1 Basic Elements

At its highest level of abstraction, a computer system is composed of three fundamental categories of components: **processors**, **memory units**, and **input/output (I/O) devices**. Each category may consist of multiple modules, and these components are interconnected through various communication pathways to fulfill the primary purpose of the system—executing programs and processing data.

To understand the architecture of a computer system, we can identify four key structural elements:

- **Processor:** The processor is the control center of the computer. It directs the execution of instructions and performs essential data processing tasks. In systems with a single processor, it is commonly referred to as the *central processing unit (CPU)*. Modern systems may include multiple processors to enhance performance and support parallel processing.
- **Main Memory:** This component temporarily stores data and instructions that are actively being used or processed. Main memory

is typically *volatile*, meaning its contents are lost when the system is powered down. It is also known as *primary memory* or *real memory*. Unlike secondary storage devices such as hard drives or SSDs, which retain data even when the system is off, main memory provides fast, temporary access to data during program execution.

- **I/O modules:** These modules manage the transfer of data between the computer and its external environment. The external environment includes a wide range of devices such as secondary storage (e.g., hard drives and SSDs), communication equipment (e.g., network interfaces), and user terminals (e.g., keyboards, monitors). I/O modules serve as intermediaries that facilitate data exchange and device control.
- **System Bus:** The system bus is the communication backbone of the computer. It connects the processor, main memory, and I/O modules, allowing them to exchange data and control signals. The bus architecture typically includes separate lines for data transfer, address specification, and control commands, ensuring synchronized and efficient communication among components.

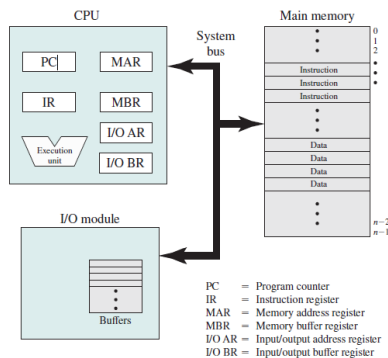


Figure 1.1: *Computer Components: Top-Level View*

Figure 1.1 shows these top-level components. A computer system is fundamentally composed of a central processing unit (CPU), main memory, and input/output (I/O) modules, all interconnected through a system bus. The CPU orchestrates the execution of instructions and data processing through specialized registers such as the Program Counter (PC), Instruction Register (IR), Memory Address Register (MAR), and Memory Buffer Register (MBR). It also manages communication with external devices using the I/O Address Register (I/O AR) and I/O Buffer Register (I/O BR), while the execution unit performs arithmetic and logic operations. Main memory serves as a temporary storage space for both instructions and data, organized into addressable blocks. It is volatile, meaning its contents are lost when the system is powered down.

The I/O module facilitates data exchange between the computer and its external environment, which includes devices such as disks, terminals,

and communication interfaces. It uses buffers to temporarily hold input and output data, allowing asynchronous operations. The system bus acts as the communication backbone, allowing data transfer and coordination among the CPU, memory, and I/O modules. It consists of data lines for transferring information, address lines for specifying locations, and control lines for managing operations. Together, these components form the structural foundation of a computer system, enabling it to execute programs and manage resources efficiently.

1.2 Microprocessors: The Foundation of Modern Computing

The revolution in computing hardware began with the invention of the **microprocessor**, a single chip that integrates the entire processing unit. Initially slower than traditional multichip processors, microprocessors have advanced significantly due to improvements in semiconductor technology. Today, they operate at sub-nanosecond speeds, enabling rapid data movement and processing.

Modern microprocessors are *multiprocessors*, meaning each chip (or socket) contains multiple **cores**. Each core is capable of executing instructions independently and includes multiple levels of **cache memory** (L1, L2, and sometimes L3) to enhance performance. Additionally, each core supports multiple **hardware threads** or **logical processors**, allowing parallel execution of tasks. For example, a quad-core processor with two threads per core can handle eight logical processors simultaneously.

1.3 GPUs: High-Performance Parallel Computing

While CPUs are optimized for general-purpose tasks, the growing demand for numerical and graphical computation has led to the rise of **Graphics Processing Units (GPUs)**. Originally designed for rendering graphics, GPUs now perform parallel computations on large datasets using **Single Instruction, Multiple Data (SIMD)** techniques.

GPUs are widely used in:

- Physics simulations and gaming
- Machine learning and AI training
- Financial modeling and spreadsheet calculations
- Real-time video processing

Their architecture includes thousands of lightweight cores, making them ideal for tasks that can be executed concurrently across many data elements.

DSPs: Specialized Signal Processing

Digital Signal Processors (DSPs) are specialized for handling continuous data streams such as audio, video, and sensor inputs. Unlike general-purpose CPUs, DSPs use optimized instruction sets and hardware accelerators for real-time signal processing.

Common applications of DSPs include:

- Audio enhancement and noise reduction
- Video encoding and decoding

- Speech recognition
- Sensor data filtering in embedded systems

DSPs were once embedded in I/O devices but are now standalone processors, especially in mobile and handheld devices.

Fixed-Function Units and Security Processors

Modern systems also include **fixed-function units**, which are dedicated hardware blocks designed for specific tasks. These units improve efficiency and reduce CPU workload.

Examples include:

- **Codecs** for audio/video compression and decompression
- **Encryption engines** for secure data transmission
- **Security processors** for authentication and cryptographic operations

These components are essential for multimedia processing and system security.

System on a Chip (SoC): Integration for Mobility

To meet the demands of compact and energy-efficient devices, the traditional microprocessor has evolved into the **System on a Chip (SoC)**. An SoC integrates multiple components—including CPUs, GPUs, DSPs, memory controllers, I/O interfaces, and sometimes main memory—onto a single chip.

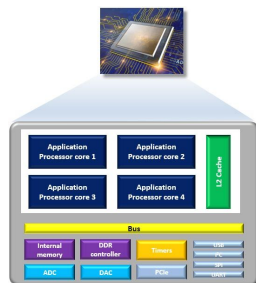


Figure 1.2: SoC Architecture - MATLAB and Simulink

1.4 Instruction Executions:

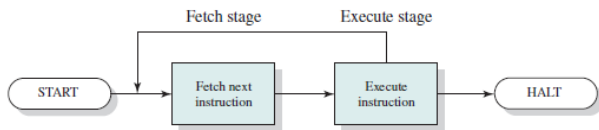


Figure 1.3: Basic Instruction Cycle

A program designed to run on a processor is composed of a series of instructions that are stored in the system's memory. The processor handles the execution of the program by following a repetitive two-step procedure: it first fetches an instruction from memory and then executes it. This repetitive operation of fetching and executing continues throughout the lifetime of the program. The process halts only when the processor is powered down, encounters a critical error that it cannot recover from, or processes a specific instruction that directs it to stop.

The complete sequence of operations required to process a single instruction is known as the **instruction cycle**. This cycle can be broken down into two main stages:

- **Fetch Stage:** The processor retrieves the next instruction from memory.
- **Execute Stage:** The processor decodes and performs the action specified by the instruction.

At the start of each instruction cycle, the processor uses the **Program Counter (PC)** to determine the address of the instruction that needs to be fetched from memory. By default, the PC increments automatically after each fetch, allowing the processor to continue fetching instructions in a sequential manner from higher memory addresses.

Once the instruction is fetched, it is placed into the **Instruction Register (IR)**. The instruction contains encoded bits that specify the operation the processor must perform. The processor then decodes the instruction and carries out the specified action. The types of actions generally fall into the following four categories:

1. **Processor-Memory Operations:** Involves transferring data between the processor and memory.
2. **Processor-I/O Operations:** Involves transferring data between the processor and an input/output (I/O) device.
3. **Data Processing Operations:** Involves performing arithmetic or logical operations on data.

4. **Control Operations:** Alters the sequence of execution. For example, the processor may modify the PC to jump to a different memory address based on the control instruction.

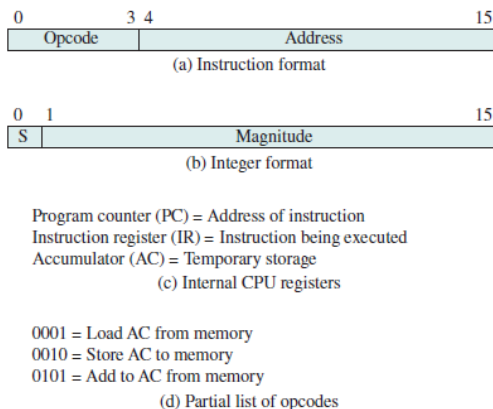


Figure 1.4: *Characteristics of a Hypothetical Machine*

Consider a simple example using a hypothetical processor that includes the characteristics listed in Figure 1.4. The processor contains a single data register, called the accumulator (AC). Both instructions and data are 16 bits long, and memory is organized as a sequence of 16-bit words. The instruction format provides 4 bits for the opcode, allowing as many as $2^4 = 16$ different opcodes (represented by a single hexadecimal digit). The opcode defines the operation the processor is to perform. With the remaining 12 bits of the instruction format, up to $2^{12} = 4,096$ (4K)

words of memory (denoted by three hexadecimal digits) can be directly addressed.

Figure 1.5 illustrates a partial program execution, showing the relevant portions of memory and processor registers. The program fragment shown adds the contents of the memory word at address 940 to the contents of the memory word at address 941 and stores the result in the latter location. Three instructions, which can be described as three fetch and three execute stages, are required:

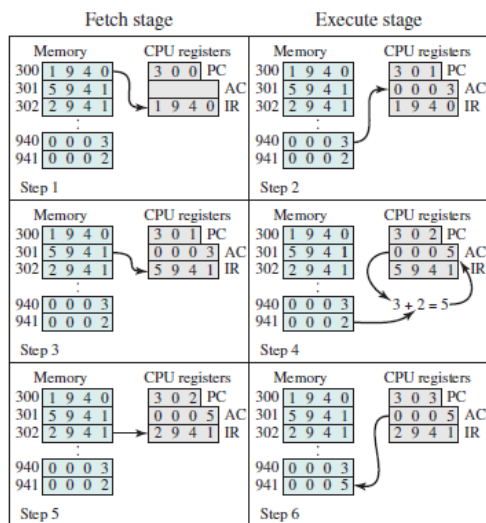


Figure 1.5: Example of Program Execution

1. The PC contains 300, the address of the first instruction. This instruction (the value 1940 in hexadecimal) is loaded into the IR and the PC is incremented. Note that this process involves the use of a memory address register (MAR) and a memory buffer register (MBR). For simplicity, these intermediate registers are not shown.
2. The first 4 bits (first hexadecimal digit) in the IR indicate that the AC is to be loaded from memory. The remaining 12 bits (three hexadecimal digits) specify the address, which is 940.
3. The next instruction (5941) is fetched from location 301 and the PC is incremented.
4. The old contents of the AC and the contents of location 941 are added, and the result is stored in the AC.
5. The next instruction (2941) is fetched from location 302, and the PC is incremented.
6. The contents of the AC are stored in location 941.

In this example, three instruction cycles, each consisting of a fetch stage and an execute stage, are needed to add the contents of location 940 to the contents of 941. With a more complex set of instructions, fewer instruction cycles would be needed. Most modern processors include instructions that contain more than one address. Thus, the execution stage for a particular instruction may involve more than one reference to memory. Also, instead of memory references, an instruction may specify an I/O operation.

Interrupts:

An **interrupt** is a signal sent to the processor that temporarily halts the current execution flow, allowing the system to respond to an event such as an I/O operation, hardware failure, or user input. Once the interrupt is handled, the processor resumes its previous activity. Interrupts are essential for efficient multitasking and responsive systems.

Classes of Interrupts

Interrupts can be categorized into several classes based on their source and purpose:

Class	Description
Hardware Interrupts	Triggered by external devices (e.g., keyboard, disk)
Software Interrupts	Generated by executing special instructions (e.g., sys
Timer Interrupts	Issued by a system timer to manage task scheduling
I/O Interrupts	Occur when an I/O device completes a task or requi
Internal Interrupts	Caused by errors like division by zero or invalid me

Program Flow of Control

Without Interrupts:

In systems without interrupts, the CPU must continuously poll devices to check their status. This leads to inefficient use of CPU time, especially during long I/O operations.

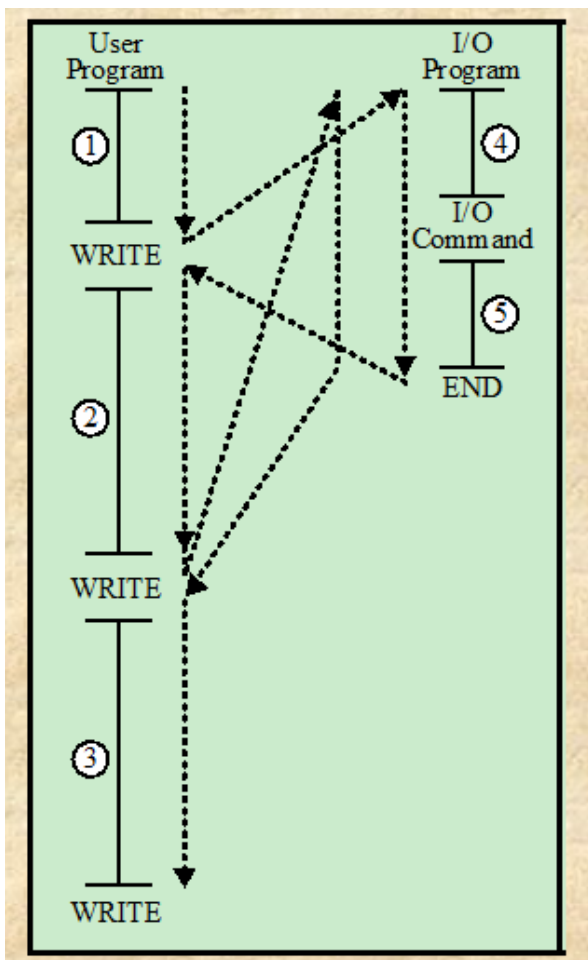


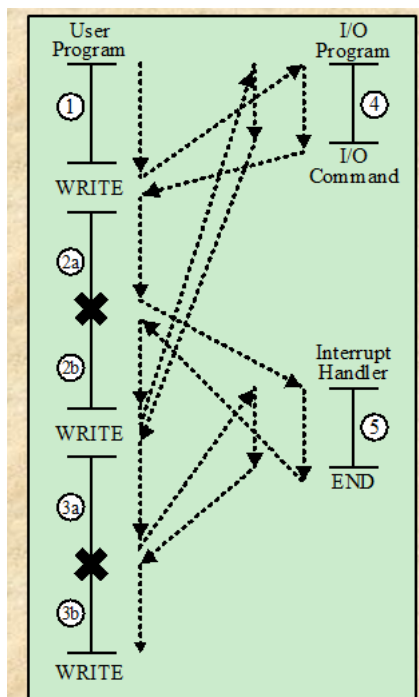
Figure 1.6: Program Flow Without Interrupts

With Interrupts:

With interrupts, the CPU can execute other tasks while waiting for I/O operations to complete. When the device is ready, it sends an interrupt signal, and the CPU temporarily switches context to handle it.

Long vs. Short I/O Interrupts:

- **Short I/O Interrupts:** These involve quick operations like keyboard input or mouse clicks. The CPU can resume its task almost immediately after handling the interrupt.
- **Long I/O Interrupts:** These involve time-consuming tasks like disk read/write or network communication. Interrupts allow the CPU to perform other operations while waiting, improving system efficiency.



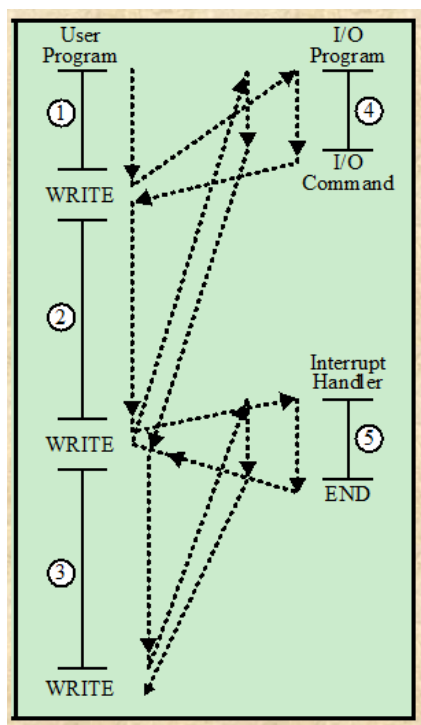


Figure 1.8: Program Flow With long I/O Interrupts

Interrupts and the Instruction Cycle

With interrupts, the processor can be engaged in executing other instructions while an I/O operation is in progress. as shown in 1.9

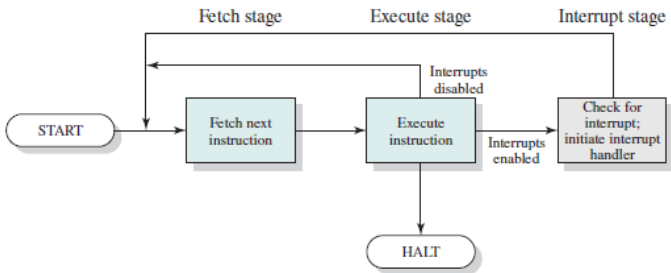


Figure 1.9: *Instruction Cycle with Interrupts*

1. **START**

The beginning of the CPU's operation. The system is powered on or reset, and the Program Counter (PC) is set to the first instruction.

2. **Fetch Stage: Fetch next instruction**

The CPU retrieves the next instruction from memory using the address in the Program Counter.

Example: If PC = 0x0040, the CPU fetches the instruction stored at that address.

3. **Execute Stage: Execute instruction**

The CPU decodes and performs the instruction. Interrupts are disabled during this stage to ensure uninterrupted execution.

Example: If the instruction is ADD R1, R2, the CPU adds the values in R1 and R2.

4. **Interrupt Stage: Check for interrupt; initiate interrupt handler**

After executing the instruction, the CPU checks if any interrupts have occurred. Interrupts are enabled here.

If an interrupt is detected:

- The CPU saves its current state (registers, PC).
- It jumps to the Interrupt Handler to deal with the event.
- After handling, it restores the state and resumes execution.

5. **Loop or HALT**

If no interrupt is found, the CPU loops back to the Fetch Stage to get the next instruction.

If a HALT instruction is encountered, the CPU stops executing further instructions.

Simplified Steps of Interrupt Handling

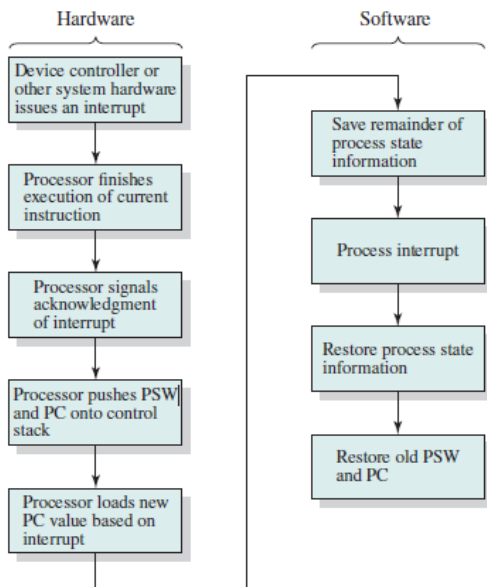


Figure 1.10: *Interrupt processing*

An interrupt triggers a number of events, the following sequence of hardware events occurs:

1. **Interrupt Request:** A device sends an interrupt signal to the processor to request attention.

2. **Complete Current Instruction:** The processor finishes executing the current instruction before responding to the interrupt.
3. **Acknowledge Interrupt:** The processor checks for pending interrupts, acknowledges the request, and the device removes its signal.
4. **Save Program State:** The processor saves the Program Counter (PC) and Program Status Word (PSW) to the control stack to resume later.
5. **Load Interrupt Handler:** The processor loads the address of the appropriate interrupt-handling routine into the PC.
6. **Save Registers and State Info:** All processor registers and other necessary state information are saved to the stack.
7. **Process the Interrupt:** The interrupt handler examines device status and performs required actions or communication.
8. **Restore Registers:** The saved register values are retrieved from the stack and restored.
9. **Resume Original Program:** The PSW and PC are restored, and the processor resumes execution of the interrupted program.

Handling Multiple Interrupts

In real-world systems, multiple devices may generate interrupts at overlapping times. For instance, a program might be receiving data from a communications line while simultaneously printing results. Each

device—such as a printer or communication controller—can generate interrupts independently. A printer may interrupt the processor after completing a print job, and a communication controller may interrupt when a unit of data arrives. These units could be individual characters or blocks, depending on the communication protocol.

It is possible for a communication interrupt to occur while a printer interrupt is already being processed. Two main strategies are used to manage such situations:

1. Disabling Interrupts During Handling

In this approach, the processor disables further interrupts while it is handling one. This means any new interrupt signals are ignored temporarily. If an interrupt occurs during this time, it remains pending and will be checked once interrupts are reenabled.

When an interrupt occurs during user program execution, interrupts are immediately disabled. After the interrupt service routine (ISR) completes, interrupts are reenabled, and the processor checks for any pending requests. This method ensures that interrupts are handled sequentially and avoids nested interrupt handling.

2. Priority-Based Interrupt Handling

This method assigns priority levels to different interrupts. If a higher-priority interrupt occurs while a lower-priority ISR is executing, the processor suspends the current ISR and switches to the higher-priority one.

Example Scenario

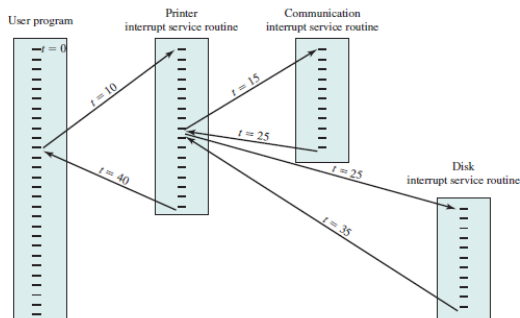


Figure 1.11: *Priority Interrupt processing*

Consider a system with three I/O devices as shown in figure:

- Printer: Priority 2
- Disk: Priority 4
- Communication Line: Priority 5

The sequence of events is as follows:

- At $t = 0$, a user program begins execution.
- At $t = 10$, a printer interrupt occurs. The processor saves the user program state and begins executing the printer ISR.

- At $t = 15$, a communication interrupt occurs. Since it has higher priority, the processor suspends the printer ISR, saves its state, and switches to the communication ISR.
- At $t = 20$, a disk interrupt occurs. Because it has lower priority than the communication ISR, it is held until the communication ISR finishes.
- At $t = 25$, the communication ISR completes. The processor resumes the printer ISR.
- Immediately at $t = 25$, the processor detects the pending disk interrupt and switches to the disk ISR before executing any printer ISR instructions.
- At $t = 35$, the disk ISR completes. The processor resumes and completes the printer ISR.
- At $t = 40$, the printer ISR finishes, and control returns to the user program.

This approach allows the system to respond to time-critical interrupts promptly, reducing the risk of data loss or buffer overflow.

1.5 *The Memory Hierarchy*

The design of a computer's memory system is influenced by several critical constraints: capacity, speed, and cost. These factors directly affect the overall performance and usability of the system.

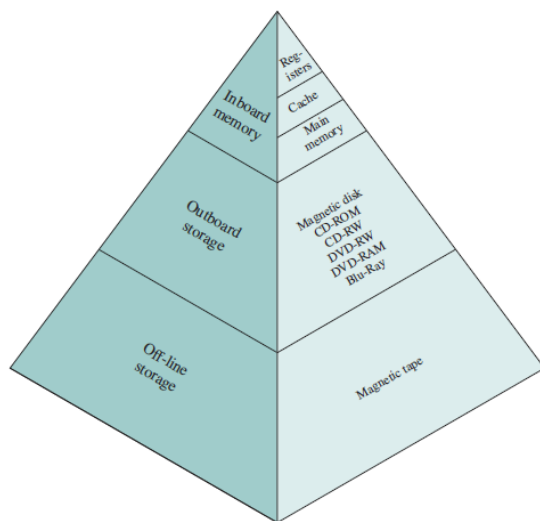


Figure 1.12: *Memory Hierarchy*

Capacity (How much?)

The amount of memory available determines how well a computer can handle applications and multitasking. Insufficient memory forces the system to rely on slower storage devices, such as hard disks or SSDs, which significantly reduces performance. On the other hand, larger memory capacity enables the execution of complex software, multitasking, and handling of large datasets efficiently. For example, modern artificial intelligence applications often require gigabytes of memory to process massive datasets.

Speed (How fast?)

Memory speed is crucial because the processor executes instructions at very high speeds. If the memory cannot keep up, the CPU wastes time waiting for data, creating a performance bottleneck. To address this, memory is organized hierarchically: small, extremely fast memory units (such as registers and cache) are placed closest to the processor, while larger but slower memory (RAM, SSD, HDD) is located further away. This hierarchy helps balance speed and size. For instance, accessing cache takes nanoseconds, while accessing a hard disk may take milliseconds.

Cost (How expensive?)

The cost of memory varies based on speed and capacity. Faster memory technologies, such as SRAM used in caches, are significantly more expensive than slower types like DRAM or magnetic storage. A practical system design balances cost and performance by combining small amounts of high-speed memory with larger amounts of slower, cheaper memory. Registers and cache are expensive but limited in size, while RAM and storage devices provide affordable large-scale capacity.

Capacity Drives Applications

When memory capacity increases, applications evolve to take advantage of the available resources. Historically, early computers with only a few kilobytes of memory could only run simple programs. In contrast, modern systems with gigabytes or terabytes of memory can run advanced software such as 3D games, real-time data analysis, and artificial intelligence workloads. This trend shows that the presence of larger memory encourages developers to create more resource-intensive applications.

Memory and Processor Speed

One of the greatest challenges in computer architecture is the gap between processor speed and memory speed. Processors are capable of executing billions of instructions per second, while memory has not advanced at the same rate. To mitigate this imbalance, architects use techniques such as caching, prefetching, and pipelining to keep the CPU supplied with data. Without these mechanisms, the CPU would spend much of its time idle, waiting for slower memory to respond.

Cost in Relation to Other Components

Memory cost must remain reasonable compared to the cost of other system components such as the CPU, GPU, and input/output devices. Overspending on memory without balancing the overall architecture leads to diminishing returns. For example, if memory accounts for the majority of a system's cost but provides only minor performance improvements, the system design is inefficient. A well-balanced computer system allocates cost across all components to maximize performance and affordability.

Principle of Locality

The principle of locality explains how programs usually use memory. It is the reason why memory hierarchies (like cache and RAM) work well. When a program is executed, the processor tends to access memory locations—both instructions and data—in groups rather than randomly. This happens because programs often contain loops and subroutines, where the same small set of instructions is repeatedly executed. Likewise, when working with arrays or tables, the processor repeatedly accesses a nearby set of data values. Although the active memory regions change

over a long period of execution, during a short time the processor mostly works within a fixed cluster of memory locations.

This behavior allows memory to be organized in a hierarchy, where higher levels handle most of the memory references. For example, in a two-level system, all program instructions and data reside in level 2 memory, while the currently needed clusters are brought temporarily into the faster level 1 memory. Occasionally, a cluster in level 1 must be swapped out to make space for a new one, but on average, most accesses are served from level 1.

The same principle applies across multiple levels of memory. At the top, the fastest, smallest, and most costly memory is found in the processor's internal registers. A typical processor contains a few dozen registers, though some advanced processors may have hundreds. Main memory, which is larger and slower, serves as the primary internal storage and provides unique addresses for instructions and data. To reduce the gap between registers and main memory, a smaller, faster cache is used. This cache, usually invisible to the programmer and processor, acts as a staging area, moving data between main memory and registers to speed up performance.

1.6 *Cache Memory*

During every instruction cycle, the processor must access memory at least once to fetch the instruction, and often additional times to retrieve operands or store results. The overall execution speed of the processor is therefore constrained by the memory cycle time—the time required to read or write a word in memory. This has long been a major issue due to the continuous gap between the rapid improvement in processor

speeds and the slower progress of main memory speeds.

This situation creates a trade-off between speed, cost, and capacity. In theory, main memory could be built using the same technology as processor registers, which would make memory access as fast as processor operations. However, this approach has always been far too costly. Instead, the practical solution is to take advantage of the principle of locality by introducing a small but very fast memory between the processor and main memory, known as the cache.

Cache Principles

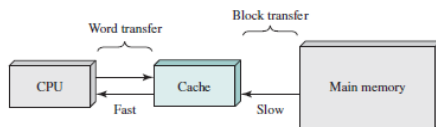


Figure 1.13: *Single Cache*

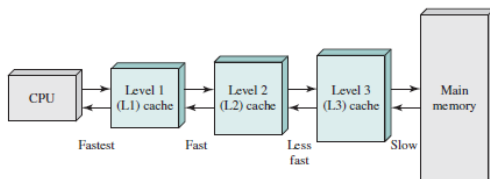


Figure 1.14: *Multilevel Cache*

Cache memory is designed to deliver access speeds close to those of the fastest memory technologies, while still allowing the use of larger and more affordable main memory built from less expensive semiconductors. As shown in Figure 1.13, the system consists of a large, slower main memory and a smaller, faster cache. The cache stores copies of portions of main memory. Whenever the processor requests a byte or word, the cache is checked first. If the requested data is found (a cache hit), it is quickly supplied to the processor. If it is not present (a cache miss), a block of data from main memory, containing a fixed number of bytes, is transferred into the cache, after which the requested item is delivered.

Thanks to the locality of reference principle, loading a block of data into the cache not only satisfies the current request but also increases the chance that upcoming memory accesses will be to nearby data already in the cache.

As illustrated in Figure 1.14, modern systems often employ multiple levels of cache. The L1 cache is the fastest but smallest, located closest to the processor. The L2 cache is larger but slower, and the L3 cache is typically even larger and slower than L2. This hierarchy balances speed and capacity to improve performance.

Cache Design

Cache design involves several important factors that directly affect system performance.

Cache Size

Even relatively small caches can greatly improve performance. The size determines how much data can be stored close to the processor.

Block Size

A block is the unit of data transferred between cache and main memory. Larger block sizes increase the chance of a cache hit due to spatial locality, since nearby data are often accessed together. However, very large blocks may reduce efficiency, as unused data may replace useful data in the cache.

Mapping Function

The mapping function determines where each block of main memory is placed in the cache. A more flexible mapping function provides better replacement options, but requires more complex hardware to search the cache and locate blocks.

Replacement Algorithm

When a new block is brought into the cache and it is already full, one block must be replaced. The *Least Recently Used (LRU)* algorithm is a common strategy, replacing the block that has not been used for the longest time. Hardware support is typically needed to track usage history.

Write Policy

The write policy defines when modified (dirty) data in the cache is written back to main memory.

- **Write-through:** Data is written to both cache and main memory immediately, ensuring consistency but increasing memory traffic.
- **Write-back:** Data is written to main memory only when the block is replaced, reducing write operations but risking temporary inconsistency.

Number of Cache Levels

Modern systems use multiple cache levels to balance speed, cost, and capacity:

- **L1 cache:** Smallest and fastest.
- **L2 cache:** Larger but slower.
- **L3 cache:** Largest and slowest, often shared among processor cores.

1.7 Direct Memory Access

There are three main techniques used for input/output (I/O) operations: **programmed I/O**, **interrupt-driven I/O**, and **direct memory access (DMA)**. Each technique differs in how the processor and I/O devices interact.

Programmed I/O

In programmed I/O, the processor directly controls the I/O operation. After sending a command, it must continuously check the device status to know when the operation is complete. This wastes processor time and slows down the system.

Interrupt-Driven I/O

With interrupt-driven I/O, the processor issues a command and continues other work. The I/O device interrupts the processor when it is ready. This is more efficient than programmed I/O, but the processor is still involved in each data transfer.

Direct Memory Access (DMA)

DMA allows large data transfers to occur directly between memory and an I/O device without involving the processor for each step. The processor only starts the transfer and is interrupted when it finishes. Although DMA may delay the processor slightly due to bus sharing, it is far more efficient for bulk data transfers.

1.8 Multiprocessor and Multicore Organization

Symmetric Multiprocessors

Definition: SMP is a computer system with **two or more similar processors** that share memory and I/O. All processors can do the **same tasks (symmetric)** and the system is controlled by one operating system.

Key Features:

- Two or more similar processors.
- Processors share the same memory and I/O.
- All processors can access I/O devices.
- All processors can perform the same functions.
- One operating system manages processors and tasks.

Advantages:

- **Performance:** Tasks can run in parallel, making the system faster.

- **Availability:** If one processor fails, the system still runs at reduced speed.
- **Incremental Growth:** Add more processors to improve performance.
- **Scaling:** Systems can be built with different numbers of processors.

Note: The benefits depend on the operating system managing parallel tasks. For the user, multiple processors are **invisible** because the OS handles scheduling and synchronization.

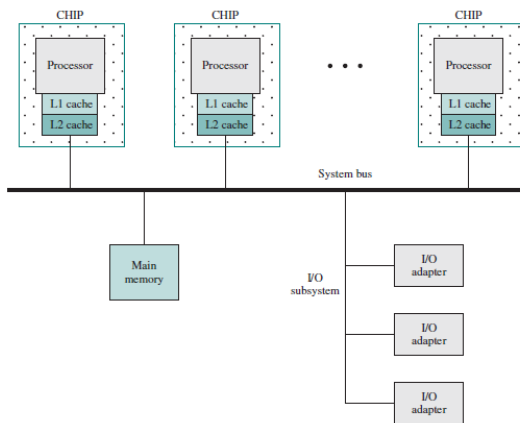


Figure 1.15: *Symmetric Multiprocessors*

Multicore Computers

A **multicore computer**, also known as a *chip multiprocessor*, combines two or more processors (*cores*) on a single piece of silicon. Each core acts like an independent processor with its own registers, ALU, control unit, pipeline hardware, and small caches (L1 instruction and data caches). Modern multicore chips also include shared caches such as L2 and sometimes L3.

The motivation for multicore systems comes from the practical limits of improving performance using higher clock speeds and more complex single-core designs. Instead, designers improved performance by placing multiple cores and larger caches on a single chip.

An example is the **Intel Core i7-5960X**, which has six cores, each with a dedicated L2 cache and a shared L3 cache. To improve efficiency, Intel uses *prefetching*, where the hardware predicts memory access patterns and loads likely-needed data into caches.

This chip also supports two forms of high-speed external communication:

- **DDR4 memory controller:** Integrated into the chip, supporting up to 64 GB/s of memory bandwidth.
- **PCI Express:** A high-speed peripheral bus providing communication between processor chips at 40 GB/s.

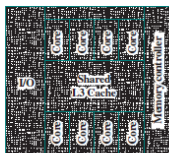
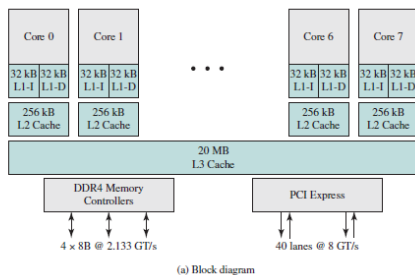


Figure 1.16: Intel Core i7-5960X Block Diagram

1.9 Exercises

1. List and briefly define the four main elements of a computer.
2. Define the two main categories of processor registers.
3. In general terms, what are the four distinct actions that a machine instruction can specify?
4. What is an interrupt?
5. How can multiple interrupts be serviced by setting priorities?

6. **What characteristics are observed while going up the memory hierarchy?**
7. **What are the trade-offs that determine the size of the cache memory?**
8. **What is the difference between a multiprocessor and a multicore system?**
9. Suppose the hypothetical processor of Figure 1.3 also has two I/O instructions:
 - 0011 = Load AC from I/O
 - 0100 = SUB from AC

In these cases, the 12-bit address identifies a particular external device. Show the program execution (using the format of Figure 1.4) for the following program:

- a) Load AC from device 7.
- b) SUB from AC contents of memory location 880.
- c) Store AC to memory location 881.

Assume that the next value retrieved from device 7 is 6 and that location 880 contains a value of 5.

True/False Questions

1. The processor controls the operation of the computer and performs its data processing functions.
2. It is not possible for a communications interrupt to occur while a printer interrupt is being processed.
3. A system bus transfers data between the computer and its external environment.
4. Cache memory is invisible to the OS.
5. With interrupts, the processor cannot be engaged in executing other instructions while an I/O operation is in progress.
6. Digital Signal Processors deal with streaming signals such as audio and video.
7. The fetched instruction is loaded into the Program Counter.
8. Interrupts are provided primarily as a way to improve processor utilization.
9. The interrupt can occur at any time and therefore at any point in the execution of a user program.
10. Over the years memory access speed has consistently increased more rapidly than processor speed.
11. An SMP can be defined as a stand-alone computer system with two or more similar processors of comparable capability.

12. The Program Status Word contains status information in the form of condition codes, which are bits typically set by the programmer as a result of program operation.
13. An example of a multicore system is the Intel Core i7.
14. In a two-level memory hierarchy the Hit Ratio is defined as the fraction of all memory accesses found in the slower memory.
15. The operating system acts as an interface between the computer hardware and the human user.

Multiple Choice Questions

1. The four main structural elements of a computer system are:
 - a) Processor, Main Memory, I/O Modules and System Bus
 - b) Processor, I/O Modules, System Bus and Secondary Memory
 - c) Processor, Registers, Main Memory and System Bus
 - d) Processor, Registers, I/O Modules and Main Memory
2. The _____ holds the address of the next instruction to be fetched.
 - a) Accumulator (AC)
 - b) Instruction Register (IR)
 - c) Instruction Counter (IC)
 - d) Program Counter (PC)

3. The..... contains the data to be written into memory and receives the data read from memory.
 - a) I/O address register
 - b) Memory address register
 - c) I/O buffer register
 - d) Memory buffer register
4. Instruction processing consists of two steps:
 - a) fetch and execute
 - b) instruction and execute
 - c) instruction and halt
 - d) fetch and instruction
5. The..... routine determines the nature of the interrupt and performs whatever actions are needed.
 - a) interrupt handler
 - b) instruction signal
 - c) program handler
 - d) interrupt signal
6. The unit of data exchanged between cache and main memory is
 - a) block size

- b) map size
 - c) cache size
 - d) slot size
7. The chooses which block to replace when a new block is to be loaded into the cache and the cache already has all slots filled with other blocks.
- a) memory controller
 - b) mapping function
 - c) write policy
 - d) replacement algorithm
8. The..... is a point-to-point link electrical interconnect specification that enables high-speed communications among connected processor chips.
- a) QPI
 - b) DDR3
 - c) LRUA
 - d) ISR
9. Small, fast memory located between the processor and main memory is called:
- a) Block memory
 - b) Cache memory

- c) Direct memory
 - d) WORM memory
10. In a uniprocessor system, multiprogramming increases processor efficiency by:
- a) Taking advantage of time wasted by long wait interrupt handling
 - b) Disabling all interrupts except those of highest priority
 - c) Eliminating all idle processor cycles
 - d) Increasing processor speed
11. The two basic types of processor registers are:
- a) User-visible and user-invisible registers
 - b) Control and user-invisible registers
 - c) Control and Status registers
 - d) User-visible and Control/Status registers
12. When an external device becomes ready to be serviced by the processor, the device sends a(n).....signal to the processor.
- a) access
 - b) halt
 - c) handler
 - d) interrupt

13. One mechanism Intel uses to make its caches more effective is, in which the hardware examines memory access patterns and attempts to fill the caches speculatively with data that is likely to be requested soon.
- a) mapping
 - b) handling
 - c) interconnecting
 - d) prefetching
14. Aorganization has a number of potential advantages over a uniprocessor organization including performance, availability, incremental growth, and scaling.
- a) temporal locality
 - b) symmetric multiprocessor
 - c) direct memory access
 - d) processor status word

Short Answer Questions

1. The invention of the _____ was the hardware revolution that brought about desktop and handheld computing.
2. To satisfy the requirements of handheld devices, the classic micro-processor is giving way to the _____.
3. The processing required for a single instruction is called a(n) _____ cycle.

4. The fetched instruction is loaded into the _____.
5. When an external device is ready to accept more data from the processor, the I/O module sends an _____ signal.

Operating System Overview

2.1 Operating System Objectives and Functions

An **Operating System (OS)** is a program that:

- Controls the execution of application programs.
- Acts as an interface between applications and hardware.

objective of Operating System

- **Convenience:** Provides a user-friendly interface to simplify interaction with the system.
- **Efficiency:** Optimizes resource usage to ensure high performance.
- **Ability to Evolve:** Supports adaptability to new technologies and changing user needs.

Operating System Services

An operating system provides a range of essential services that support both user applications and system-level operations. One of its primary roles is **program development**, offering tools such as editors, compilers, and debuggers to facilitate software creation. Once developed, the OS handles **program execution** by loading programs into memory, managing CPU scheduling, and overseeing process control. It also enables **access to I/O devices**, abstracting hardware complexities through device drivers and system calls to ensure smooth interaction with peripherals like keyboards, printers, and storage devices. Another critical service is **controlled access to files**, where the OS manages permissions and enforces security policies to protect data integrity. Through **system access**, the OS authenticates users and regulates their privileges, safeguarding system resources from unauthorized use. Additionally, the OS performs **error detection and response**, continuously monitoring operations to identify faults and initiate recovery procedures. Lastly, **accounting** services track resource usage, allowing for performance analysis, auditing, and billing in multi-user environments. Together, these services form the backbone of a reliable and efficient computing environment.

The Operating System as a User/Computer Interface

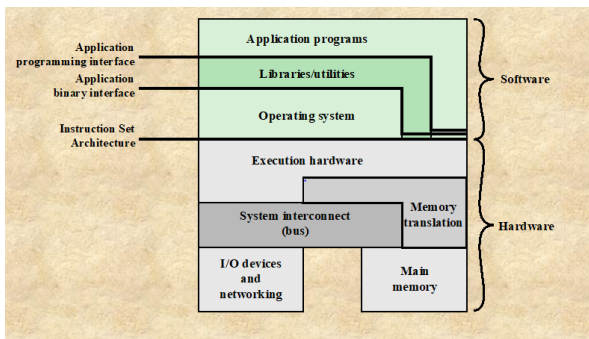


Figure 2.1: Hardware and software structure

Modern computing systems rely on several layers of abstraction to ensure compatibility, performance, and portability. The three interfaces highlighted—ISA, ABI, and API—play distinct but interconnected roles in bridging hardware and software.

2.2 Instruction Set Architecture (ISA)

- **Definition:** ISA defines the set of machine-level instructions that a processor can execute.
- **Role:** It serves as the boundary between hardware and low-level software (like compilers and operating systems).
- **Examples:** x86, ARM, RISC-V.

- **Impact:** Determines how software must be written to run on a specific processor. Changes in ISA often require recompilation or redesign of software.

2.3 *Application Binary Interface (ABI)*

- **Definition:** ABI specifies the binary format, calling conventions, and system call interface for a given OS and hardware architecture.
- **Role:** It ensures that compiled applications can run correctly on a system by defining how functions are called, how data is represented, and how system resources are accessed.
- **Examples:** System V ABI for UNIX systems, Windows x64 ABI.
- **Impact:** Critical for binary compatibility. Developers targeting a specific ABI can distribute compiled binaries without needing source code recompilation.

2.4 *Application Programming Interface (API)*

- **Definition:** API defines a set of functions and protocols that developers use to build applications for a specific operating system or platform.
- **Role:** It abstracts system-level operations (like file access, networking, or GUI interactions) into callable functions.
- **Examples:** POSIX API, Windows API, Java API.

- **Impact:** Facilitates software development by providing reusable building blocks. APIs also enable portability across platforms if standardized.

The Operating System as Resource Manager

The Operating System (OS) serves as a central control mechanism for managing a computer's resources, including input/output devices, memory, and processor time. Unlike traditional control systems that are external to the entities they regulate, the OS is embedded within the system it manages. This internal control structure introduces unique characteristics in how the OS operates and interacts with hardware.

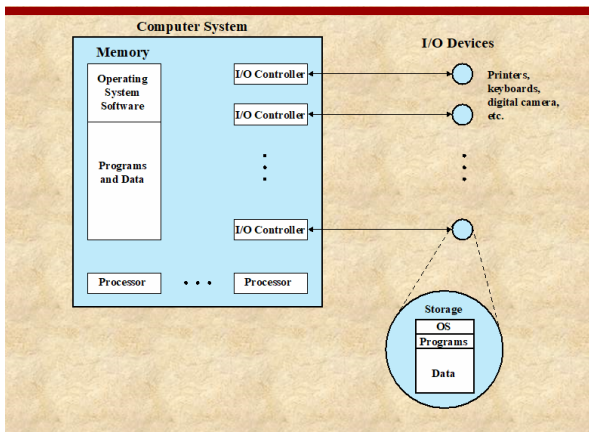


Figure 2.2: *The Operating System as Resource Manager*

Main parts of operating system as shown in figure 2.2

- **Embedded Control Mechanism:**

- The OS is not a separate entity; it is a software program executed by the processor.
- It controls resources from within the system, unlike external controllers such as thermostats.

- **Execution and Control Dynamics:**

- The OS must frequently relinquish control to allow the processor to execute user programs.
- It regains control intermittently to manage and prepare resources for subsequent tasks.

- **Processor Dependency:**

- The OS makes decisions about resource allocation.
- The processor must stop executing the OS to carry out these decisions by running other programs.

- **Resource Management Responsibilities:**

- **Main Memory:**

- * Contains the OS kernel and active components.
- * Also holds user programs, utilities, and data.
- * Memory allocation is jointly managed by the OS and processor hardware.

– I/O Devices:

- * OS decides when devices can be accessed by executing programs.
- * Controls file access and usage.

– Processor Time:

- * The processor is treated as a resource.
- * OS determines time allocation for user program execution.

2.5 The Evolution of Operating Systems

In attempting to understand the key requirements for an OS and the significance of the major features of a contemporary OS, it is useful to consider how operating systems have evolved over the years.

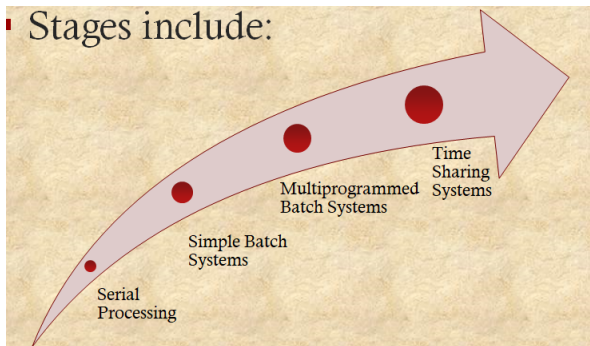


Figure 2.3: *Evolution of Operating Systems*

Serial Processing

During the late 1940s to mid-1950s, computers operated without any form of operating system. Programmers interacted directly with the machine hardware using physical consoles equipped with display lights, toggle switches, input devices such as card readers, and output devices like printers. Programs were written in raw machine code and manually loaded into memory. If a program encountered an error, the console lights would indicate the fault. Successful execution would result in printed output. This process was entirely manual and highly susceptible to human error.

Scheduling Challenges

Computer usage followed a strictly serial model, where users accessed the machine one at a time. Scheduling was managed manually through hardcopy sign-up sheets. Users reserved time slots—typically in half-hour increments—but this system was inefficient. If a user completed their task early, the remaining time was wasted. Conversely, if a user ran into issues and exceeded their time slot, they were often forced to stop before resolving the problem. This rigid scheduling led to significant underutilization of computing resources.

Setup Time and Operational Overhead

Running a single program, or *job*, involved multiple setup stages. First, the compiler had to be loaded into memory, followed by the source code written in a high-level language. After compilation, the resulting object code was saved, and then linked with common functions to produce the final executable. Each of these steps required manual intervention, such as mounting tapes or organizing card decks. If an error occurred at

any point, the user typically had to restart the entire setup process from scratch. This repetitive and time-consuming procedure significantly hindered productivity.

Serial Processing Model

The operational mode of these early systems is best described as *serial processing*. Only one user could interact with the computer at a time, and tasks were executed sequentially. There was no concept of multitasking or concurrent execution. This model, while simple, was inefficient and failed to maximize the potential of the hardware. The lack of automation and system-level coordination meant that valuable computing time was frequently lost due to human delays and procedural bottlenecks.

Simple Batch Systems

Early computers were extremely costly, so maximizing processor utilization was essential. Idle time caused by scheduling and job setup was considered wasteful. To solve this, the concept of the **Batch Operating System (Batch OS)** emerged in the 1950s. The first was built by General Motors for the IBM 701, later refined for the IBM 704. By the 1960s, many vendors had their own batch OS, with **IBSYS (IBM 7090/7094)** being especially influential.

Basic Concept of Batch Processing

The main idea was the introduction of the **monitor program**, which controlled execution. Users submitted jobs on **cards or tape** to an operator, who grouped them into batches and loaded them into the computer. The monitor ensured each job executed in sequence, and once a job completed, *control returned to the monitor*, which automatically loaded the next job. Results were sent to output devices such as printers.

Operation from Two Perspectives

Monitor's Perspective: The monitor controlled the order of events. A part of it, called the **resident monitor**, always remained in main memory. It loaded jobs one by one from input, placed them into the user program area, and executed them until completion, after which the next job was loaded.

Processor's Perspective: The processor alternated between executing monitor instructions and user program instructions. “Passing control to a job” meant executing user instructions, while “returning control to the monitor” meant resuming execution of monitor code.

Features and Functions

The monitor provided **scheduling**, ensuring jobs were executed without idle time, and supported a primitive **Job Control Language (JCL)** that let users specify instructions for job handling.

To make batch OS practical, certain **hardware features** were required:

- **Memory Protection:** Prevented user programs from altering monitor memory.
- **Timer:** Ensured no single job monopolized the processor.
- **Privileged Instructions:** Reserved for the monitor (e.g., I/O operations).
- **Interrupts:** Allowed the OS to gain or relinquish control more flexibly.

Modes of Operation

These hardware considerations introduced the idea of **two modes**:

- **User Mode:** Restricted access to protected memory and privileged instructions.
- **System/Kernel Mode:** Allowed full access for the monitor.

This separation ensured stability and prevented chaos in execution.

Trade-Offs and Benefits

Batch OS introduced overhead because part of main memory and processor time was consumed by the monitor. However, despite this cost, batch processing **significantly improved system utilization** compared to manual job setup and execution.

Multiprogrammed Batch Systems

A **Multiprogrammed Batch System** is an early operating system model designed to improve CPU utilization by allowing multiple jobs to reside in memory simultaneously. Unlike simple batch systems that execute one job at a time, multiprogramming enables the system to switch between jobs whenever one is waiting for I/O operations, thereby reducing idle processor time.

the Key Characteristics

- **Multiple Jobs in Memory** The system maintains several jobs in main memory at once. The operating system selects and schedules jobs based on resource availability and execution readiness.
- **CPU and I/O Overlap** When a job initiates an I/O operation, which is typically slow, the CPU can switch to another job that is ready to execute. This overlap ensures that the CPU remains active and productive.

- **Automatic Job Sequencing** The operating system automates the selection and sequencing of jobs, eliminating the need for manual intervention and improving overall system efficiency.

Operational Workflow

1. **Job Loading:** Multiple jobs are loaded into memory.
2. **Execution Begins:** The CPU starts executing one job.
3. **I/O Wait:** If the job requests I/O, the CPU switches to another job.
4. **Job Switching:** The OS manages transitions to keep the CPU busy.
5. **Completion:** Completed jobs are removed from memory; new jobs may be loaded.

Problems Solved

Multiprogramming addresses several inefficiencies found in earlier serial systems. The following table summarizes these improvements:

Problem in Serial Systems	Multiprogramming Solution
Processor is often idle	Keeps CPU busy by switching to ready jobs
I/O devices are slow compared to CPU	Allows CPU to work on other jobs during I/O waits
Manual job sequencing	Automates job selection and execution

Limitations

- No user interaction; designed for batch processing only.
- Requires complex scheduling algorithms.
- Limited memory may restrict the number of concurrent jobs.

Time-Sharing Systems

Time-sharing systems are designed to allow multiple users to interact with a computer system simultaneously. The operating system manages processor time efficiently to ensure responsiveness and fairness.

- **Supports Multiple Interactive Jobs:** Time-sharing systems can handle several user programs at once, making them suitable for interactive environments.
- **Shared Processor Time:** The CPU time is divided into small units (time slices or quanta) and distributed among active users, enabling concurrent execution.
- **Simultaneous Access via Terminals:** Multiple users connect through terminals. The operating system interleaves the execution of each user's program, switching rapidly between tasks to simulate parallelism.

Compatible Time-Sharing System (CTSS):

The Compatible Time-Sharing System (CTSS) was one of the earliest time-sharing operating systems, developed at MIT by a research group known as Project MAC. First implemented in 1961 for the IBM 709,

CTSS operated on a machine equipped with 32,000 36-bit words of main memory, with 5,000 words reserved for the resident monitor. CTSS introduced a novel technique called *time slicing*, where the system clock generated interrupts approximately every 0.2 seconds. At each interrupt, the operating system regained control and reassigned the processor to another user, enabling multiple users to interact with the system concurrently. To facilitate this, user programs and data were written to disk before switching and restored into main memory when the user's turn resumed. This mechanism allowed efficient multitasking and laid the groundwork for future developments in interactive computing.

2.6 Major Achievements

Operating Systems are among the most complex pieces of software ever developed. Major advances in development include:

- Process
- Memory Management
- Information Protection and Security
- Scheduling and Resource Management
- Structure

The Process

Many definitions have been given for the term process, including

- A program in execution

- An instance of a program running on a computer
- The entity that can be assigned to and executed on a processor
- A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources

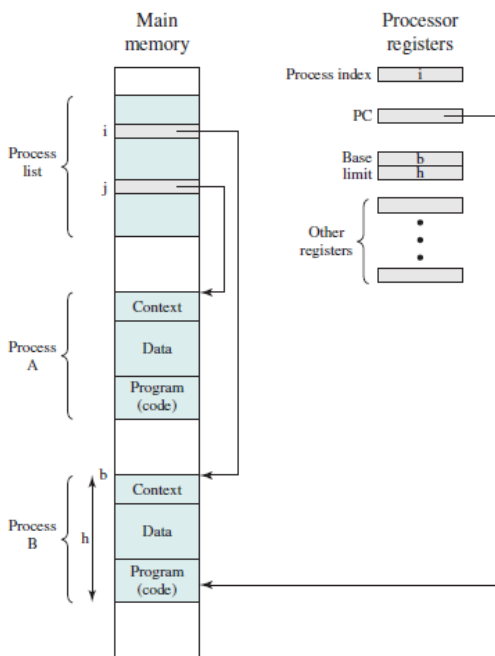
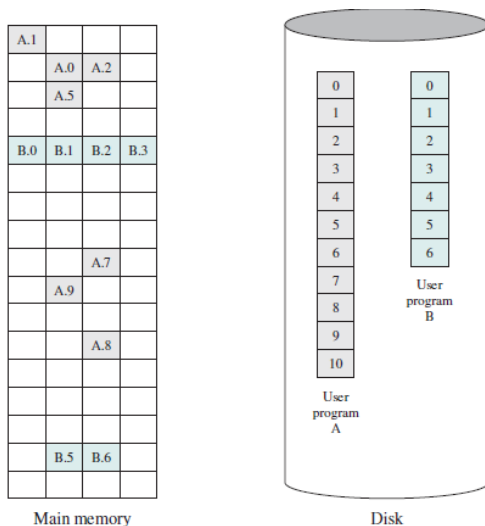


Figure 2.4: *Process implementation*

figure 2.4 indicates a way in which processes may be managed. Two processes, A and B, exist in portions of main memory. That is, a block of memory is allocated to each process that contains the program, data, and context information. Each process is recorded in a process list built and maintained by the OS. The process list contains one entry for each process, which includes a pointer to the location of the block of memory that contains the process. The entry may also include part or all of the execution context of the process. The remainder of the execution context is stored elsewhere, perhaps with the process itself (as indicated in Figure 2.4) or frequently in a separate region of memory. The process index register contains the index into the process list of the process currently controlling the processor. The program counter points to the next instruction in that process to be executed. The base and limit registers define the region in memory occupied by the process: The base register is the starting address of the region of memory and the limit is the size of the region (in bytes or words). The program counter and all data references are interpreted relative to the base register and must not exceed the value in the limit register. This prevents interprocess interference.

In Figure 2.4, the process index register indicates that process B is executing. Process A was previously executing but has been temporarily interrupted. The contents of all the registers at the moment of A's interruption were recorded in its execution context. Later, the OS can perform a process switch and resume execution of process A. The process switch consists of storing the context of B and restoring the context of A. When the program counter is loaded with a value pointing into A's program area, process A will automatically resume execution.

Figure 2.5: *virtual memory concept*

Memory Management

The operating system (OS) is responsible for managing memory through five principal storage management functions. First, it provides **process isolation**, ensuring that each process operates independently without interfering with others, thereby maintaining system integrity and security. Second, the OS performs **automatic allocation and management**, dynamically assigning memory to processes and reclaiming it when no longer needed. Third, it supports **modular programming**, enabling programs to be structured into reusable and independently loadable modules. Fourth, the OS enforces **protection and access control**, re-

stricting access to memory regions to authorized processes only. Finally, it manages **long-term storage**, preserving data across sessions and system reboots. These responsibilities are essential for efficient, secure, and organized memory utilization in modern computing environments.

Virtual Memory

Virtual memory is a facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available. Virtual memory was conceived to meet the requirement of having multiple user jobs reside in main memory concurrently, so that there would not be a hiatus between the execution of successive processes while one process was written out to secondary store and the successor process was read in.

Paging

Because processes vary in size, if the processor switches among a number of processes it is difficult to pack them compactly into main memory. Paging systems were introduced, which allow processes to be comprised of a number of fixed-size blocks, called pages. A program references a word by means of a virtual address consisting of a page number and an offset within the page. Each page of a process may be located anywhere in main memory. The paging system provides for a dynamic mapping between the virtual address used in the program and a real address , or physical address, in main memory.

Information Protection and Security

Information protection and security are critical concerns for any organization, and the nature of threats can vary widely depending on specific circumstances. The core challenge lies in controlling access to

computer systems and safeguarding the information stored within them. Effective security strategies must address several key issues: ensuring the availability of systems and data so that authorized users can access resources when needed; maintaining confidentiality to prevent unauthorized disclosure of sensitive information; preserving data integrity to ensure that information remains accurate and unaltered; and verifying authenticity to confirm that users and data are genuine and trustworthy. Together, these elements form the foundation of a robust information security framework.

Scheduling and Resource Management

Scheduling and Resource Management highlights a fundamental responsibility of operating systems: managing system resources effectively. It emphasizes that resource allocation policies must be carefully designed to balance three critical factors. First, fairness, which ensures that all processes receive equitable access to resources without bias. Second, efficiency, which aims to maximize system performance by minimizing waste and optimizing resource usage. Third, differential responsiveness, which allows the system to prioritize tasks based on urgency or importance, ensuring timely execution of critical processes. Together, these principles guide how an OS schedules tasks and allocates resources to maintain stability, performance, and user satisfaction.

2.7 Developments Leading to Modern Operating Systems

The rate of change in the demands on operating systems requires not just modifications and enhancements to existing architectures but new

ways of organizing the OS.

A wide range of different approaches and design elements has been tried in both experimental and commercial operating systems, but much of the work fits into the following categories:

- Microkernel architecture
- Multithreading
- Symmetric multiprocessing
- Distributed operating systems
- Object-oriented design

Microkernel architecture

Microkernel architecture represents a minimalist design philosophy in operating systems, where only the most fundamental services are implemented within the kernel. Unlike monolithic kernels that incorporate a wide array of functionalities, microkernels aim to enhance modularity, maintainability, and system reliability by delegating non-essential services to user space.

Core Kernel Responsibilities

In a microkernel-based system, the kernel is responsible for a limited set of essential functions:

- **Address Space Management:** Handles memory allocation and protection, ensuring that processes operate within their designated memory regions.

- **Interprocess Communication (IPC):** Facilitates communication between processes, typically through message passing mechanisms.
- **Basic Scheduling:** Manages the execution order of processes to ensure fair and efficient CPU utilization.

Architectural Advantages

The microkernel approach offers several notable benefits:

- **Simplifies Implementation:** With fewer responsibilities, the kernel becomes easier to develop, test, and maintain.
- **Provides Flexibility:** System services can be implemented in user space, allowing for easier updates and customization.
- **Well Suited to Distributed Environments:** Its modular nature supports distributed systems where components may operate across multiple machines.

Multithreading

Multithreading is a technique in which a process is divided into multiple threads that can execute concurrently. This approach enhances the performance and responsiveness of applications by allowing multiple tasks to be performed simultaneously within the same process.

Definition of a Thread

A **thread** is the smallest unit of execution within a process. Unlike processes, threads share the same memory space and system resources, making them lightweight and efficient. Each thread includes:

- A program counter, stack, and set of registers.
- Access to the shared memory and resources of the parent process.
- Independent execution capability, allowing concurrent operations.

Benefits of Multithreading

Multithreading offers several advantages in application design and system performance:

- **Improved Responsiveness:** Threads can handle user input, background tasks, and I/O operations simultaneously.
- **Resource Sharing:** Threads within the same process share memory and resources, reducing overhead.
- **Efficient CPU Utilization:** Multiple threads can run on multiple cores, maximizing parallelism.
- **Simplified Program Structure:** Complex tasks can be broken into simpler, concurrent threads.

Thread vs. Process

Thread	Process
Lightweight unit of execution within a process	Independent execution unit with its own memory space
Shares memory and resources with other threads in the same process	Has its own memory and system resources
Faster context switching and lower overhead	Higher overhead due to memory isolation
Used for fine-grained parallelism	Used for coarse-grained task separation

Table 2.1: Comparison between Threads and Processes

Symmetric multiprocessing

Symmetric Multiprocessing (SMP) refers to a computer hardware architecture where two or more identical processors are connected to a single shared main memory and operate under a single operating system. The term also encompasses the OS behavior that effectively utilizes this architecture.

In an SMP system, the operating system is responsible for:

- **Scheduling Processes and Threads:** The OS distributes workloads evenly across all available processors.
- **Exploiting Parallelism:** It must provide mechanisms and tools to take advantage of concurrent execution.

- **Transparency to the User:** The user does not need to manage processor allocation; the OS handles it seamlessly.

SMP vs. Multithreading

While SMP and multithreading are often discussed together, they are distinct concepts:

- **SMP** is a hardware and OS-level feature that enables multiple processors to execute tasks concurrently.
- **Multithreading** is a software-level technique that allows a single process to be divided into multiple threads.
- SMP can enhance multithreaded applications by allowing threads to run truly in parallel on separate processors.

Advantages of SMP

- **Improved Performance:** Tasks can be executed in parallel, reducing execution time.
- **Scalability:** Additional processors can be added to improve throughput.
- **User Transparency:** The complexity of processor management is hidden from the user, simplifying application development.

Operating System Design Paradigms

Operating system (OS) design encompasses the architectural principles and methodologies used to build efficient, scalable, and maintainable systems. Two notable paradigms in modern OS design are the **Distributed**

Operating System and **Object-Oriented Design**. Each offers distinct advantages and targets different system requirements.

Distributed Operating System

A **Distributed Operating System** provides the illusion of a unified system across multiple machines. It integrates:

- A single main memory space and a secondary memory space.
- Unified access to distributed resources such as file systems and devices.
- Seamless interaction across networked nodes, appearing as one coherent system to the user.

Challenges include:

- Performance lag compared to tightly coupled systems like uniprocessor or SMP-based OS.
- Complexity in synchronization, fault tolerance, and resource sharing.

Object-Oriented Design

Object-Oriented Design introduces modularity and encapsulation into OS development. Key features include:

- **Modular Extension:** Facilitates disciplined addition of new functionalities to a small kernel.

- **Customization:** Allows programmers to tailor OS behavior without compromising system integrity.
- **Tool Development:** Supports the creation of distributed tools and full-fledged distributed systems with reusable components.

Comparative Insights

Distributed Operating System	Object-Oriented Design
Focuses on resource sharing across networked systems	Focuses on modularity and encapsulation in OS structure
Provides a unified system image across multiple nodes	Enables safe and flexible extension of OS functionality
Challenges include latency and synchronization across distributed components	Simplifies development of scalable and maintainable OS components
Ideal for large-scale distributed environments	Ideal for customizable and extensible OS architectures

Table 2.2: *Comparison of OS Design Paradigms*

2.8 Exercises

1. What are three objectives of an OS design?
2. What is the kernel of an OS?
3. What is multiprogramming?

4. What is a process?
5. List and briefly explain five storage management responsibilities of a typical OS.
6. What is time slicing?
7. Explain the difference between a monolithic kernel and a micro-kernel.
8. What is multithreading?
9. What do you understand by a distributed operating system?

True/False Questions

1. An OS should be constructed in such a way as to permit the effective development, testing, and introduction of new system functions without interfering with service.
2. The OS masks the details of the hardware from the programmer and provides the programmer with a convenient interface for using the system.
3. The ABI gives a program access to the hardware resources and services available in a system through the user ISA.
4. The OS frequently relinquishes control and must depend on the processor to allow it to regain control.
5. One of the driving forces in operating system evolution is advancement in the underlying hardware technology.

6. The processor itself is not a resource so the OS is not involved in determining how much of the processor time is devoted to the execution of a user program.
7. A process consists of three components: an executable program, the associated data needed by the program, and the execution context of the program.
8. Uniprogramming typically provides better utilization of system resources than multiprogramming.
9. A monolithic kernel is implemented as a single process with all elements sharing the same address space.
10. The user has direct access to the processor with a batch-processing type of OS.
11. Both batch processing and time-sharing use multiprogramming.
12. The phrase “control is passed to a job” means that the processor is now fetching and executing instructions from the monitor program.
13. In a time-sharing system, a user’s program is preempted at regular intervals, but due to relatively slow human reaction time this occurrence is usually transparent to the user.
14. The principle objective of Batch Multiprogramming is to minimize response time.

15. Virtualization technology enables a single PC or server to simultaneously run multiple operating systems or multiple sessions of a single OS.

Multiple Choice Questions

For each question, select the best answer.

1. The _____ is the interface that is the boundary between hardware and software.
 - a) ABI
 - b) ISA
 - c) IAS
 - d) API
2. A(n)..... is a set of resources for the movement, storage, and processing of data and for the control of these functions.
 - a) architecture
 - b) program
 - c) computer
 - d) application
3. The operating system's..... refers to its inherent flexibility in permitting functional modifications to the system without interfering with service.
 - a) efficiency

- b) ability to evolve
- c) controlled access
- d) convenience

4. Operating systems must evolve over time because:

- a) new hardware is designed and implemented in the computer system
- b) hardware must be replaced when it fails
- c) hardware is hierarchical
- d) users will only purchase software that has a current copyright date

5. A special type of programming language used to provide instructions to the monitor is.....

- a) FPL
- b) JCL
- c) DML
- d) SML

6. Hardware features desirable in a batch-processing operating system include memory protection, timer, privileged instructions, and

- a) clock cycles
- b) associated data

- c) interrupts
 - d) kernels
7. A user program executes in a , in which certain areas of memory are protected from the user's use, and in which certain instructions may not be executed.
- a) kernel mode
 - b) user mode
 - c) task mode
 - d) batch mode
8. Multiprogramming operating systems are fairly sophisticated compared to single-program or systems.
- a) uniprogramming
 - b) time-sharing
 - c) multitasking
 - d) memory management
9. One of the first time-sharing operating systems to be developed was the
- a) Compatible Time-Sharing System
 - b) Real Time Transaction System
 - c) Multiple-Access System
 - d) Multiprogramming Operation System

10. The technique where a system clock generates interrupts, and at each clock interrupt the OS regains control and assigns the processor to another user, is
- a) time slicing
 - b) multithreading
 - c) round robin
 - d) clock cycle
11. The is the internal data by which the OS is able to supervise and control the process.
- a) executable program
 - b) associated data
 - c) nucleus
 - d) execution context
12. is where the OS must prevent independent processes from interfering with each other's memory, both data and instructions.
- a) Support of modular programming
 - b) Process isolation
 - c) Automatic allocation and management
 - d) Protection and access control
13. is concerned with the proper verification of the identity of users and the validity of messages or data.

- a) Availability
 - b) Confidentiality
 - c) Authenticity
 - d) Data integrity
14. A common strategy to give each process in the queue some time in turn is referred to as a technique.
- a) multithreading
 - b) round-robin
 - c) time slicing
 - d) serial processing

Chapter 3

Process Description and Control

3.1 What Is a Process?

Background

A computer platform consists of hardware resources such as the processor, memory, and I/O devices, while applications are developed to perform tasks by taking input, processing it, and producing output. Writing applications directly for hardware is inefficient because common routines are needed to access resources, hardware alone provides limited support for multiprogramming, and applications must be protected from interfering with one another. To address this, the operating system (OS) was developed as a software layer between applications and hardware, providing a secure, convenient, and consistent interface. The OS offers uniform abstractions of resources like memory, file systems, and networks, while managing their sharing and protection. Its key goals are to make resources available to multiple applications, switch the processor among them to ensure progress, and use the CPU and I/O devices efficiently. Modern operating systems achieve this by managing application execution through processes.

Processes and Process Control Blocks

A process can be defined in several ways: as a program in execution, an instance of a program running on a computer, an entity assigned to and executed on a processor, or a unit of activity consisting of an instruction sequence, a current state, and associated system resources. Essentially, a process includes program code (which may be shared by multiple processes) and a set of data linked to that code. While executing, a process can be uniquely described by specific elements: an identifier to distinguish it from others, its state (e.g., running, ready, or blocked), a priority level, a program counter pointing to the next instruction, memory pointers for code, data, and shared blocks, context data stored in processor registers, I/O status information (such as pending requests, assigned devices, and files in use), and accounting information (like processor time, limits, and account numbers). All of this information is maintained in a Process Control Block (PCB), a data structure created and managed by the operating system. The PCB is crucial because it stores enough information to suspend and later resume a process without loss of continuity. When a process is interrupted, its program counter and register contents (context data) are saved in its PCB, and its state is updated. The OS can then load the context of another process into the processor, allowing it to run. Thus, a process can be seen as program code + associated data + its PCB, and on a single-processor system, only one process can be in the running state at a given time.

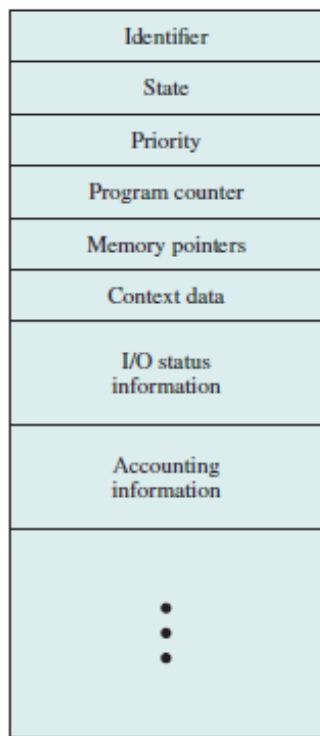


Figure 3.1: *Process Control Block*

3.2 *Process States and life Cycle*

When a program is set to run, the operating system creates a process—also known as a task—to manage its execution. This process acts as

a container for the program's instructions, data, and execution context.

From the processor's perspective, its job is to execute instructions from its internal set. It does this by following a sequence determined by the program counter (PC), a special register that holds the memory address of the next instruction to execute. As the processor runs, the value in the PC changes, guiding it through the program's code.

However, over time, the PC might point to instructions from different programs—each belonging to a separate process. This happens because the processor switches between processes, especially in multitasking systems. So, while the processor sees a continuous stream of instructions, these instructions may come from multiple sources.

From the individual program's point of view, its execution is simply a linear sequence of its own instructions. This sequence can be recorded and analyzed as a trace, which is essentially a log of all instructions executed by that process.

To understand how the processor behaves overall, we look at how these individual traces from different processes are interleaved—that is, how the processor switches between them over time. This interleaving reflects the multitasking nature of modern systems.

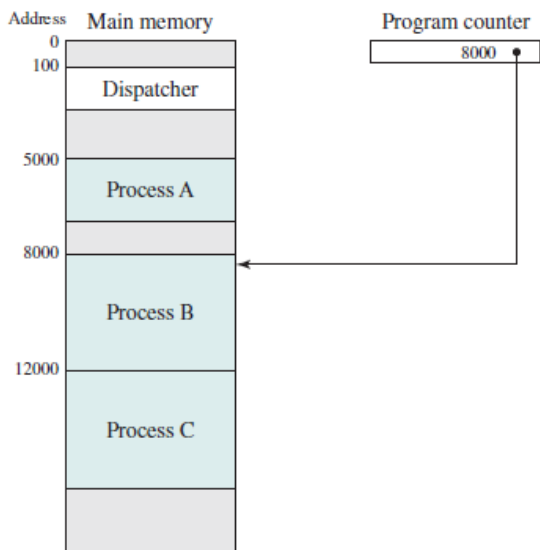


Figure 3.2: *Example of Execution*

5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004		12004
5005		12005
5006		12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

(a) Trace of process A

(b) Trace of process B

(c) Trace of process C

5000 = Starting address of program of process A

8000 = Starting address of program of process B

12000 = Starting address of program of process C

Figure 3.3: *Traces of processes*

Let us consider a very simple example. Figure 3.2 shows a memory layout of three processes. To simplify the discussion, we assume no use of virtual memory; thus all three processes are represented by programs that are fully loaded in main memory. In addition, there is a small **dispatcher program** that switches the processor from one process to another. Figure 3.3 shows the traces of each of the processes during the early part of their execution. The first 12 instructions executed in processes A and C are shown. Process B executes four instructions, and we assume the fourth instruction invokes an I/O operation for which the process must wait.

1	5000	27	12004
2	5001	28	12005
3	5002	-----Time-out	
4	5003	29	100
5	5004	30	101
6	5005	31	102
-----Time-out		32	103
7	100	33	104
8	101	34	105
9	102	35	5006
10	103	36	5007
11	104	37	5008
12	105	38	5009
13	8000	39	5010
14	8001	40	5011
15	8002	-----Time-out	
16	8003	41	100
-----I/O request		42	101
17	100	43	102
18	101	44	103
19	102	45	104
20	103	46	105
21	104	47	12006
22	105	48	12007
23	12000	49	12008
24	12001	50	12009
25	12002	51	12010
26	12003	52	12011
		-----Time-out	

100 = Starting address of dispatcher program

Shaded areas indicate execution of dispatcher process;

first and third columns count instruction cycles;

second and fourth columns show address of instruction being executed.

Figure 3.4: *combined Traces of processes*

Now let us view these traces from the processor's point of view. Figure 3.4 shows the interleaved traces resulting from the first 52 instruction cycles (for convenience, the instruction cycles are numbered). In this figure, the shaded areas represent code executed by the dispatcher. The same sequence of instructions is executed by the dispatcher in each instance because the same functionality of the dispatcher is being exe-

cuted. We assume the OS only allows a process to continue execution for a maximum of six instruction cycles, after which it is interrupted; this prevents any single process from monopolizing processor time. As Figure 3.4 shows, the first six instructions of process A are executed, followed by a time-out and the execution of some code in the dispatcher, which executes six instructions before turning control to process B. After four instructions are executed, process B requests an I/O action for which it must wait. Therefore, the processor stops executing process B and moves on, via the dispatcher, to process C. After a time-out, the processor moves back to process A. When this process times out, process B is still waiting for the I/O operation to complete, so the dispatcher moves on to process C again.

A Two-State Process Model

The operating system's main job is to control process execution by deciding which process runs and how resources are shared. In the simplest model, a process can be in only two states: Running (currently using the CPU) or Not Running (waiting for CPU time). When the OS creates a process, it makes a process control block (PCB) and places the process in the Not Running state. The dispatcher in the OS then switches between processes by moving the current one from Running to Not Running, and selecting another process to start running.

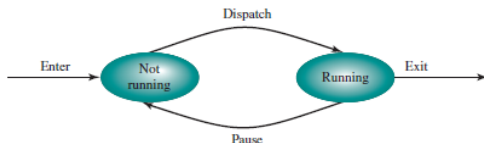


Figure 3.5: *state transition diagram*

Here's the two-state process model diagram:

1. The operating system creates a new process by generating a *Process Control Block (PCB)* and placing the process in the **Not Running** state.
2. The dispatcher selects a process from the **Not Running** state and moves it to the **Running** state to use the CPU.
3. While executing, the process may be **interrupted** (e.g., by the OS or an event).
4. When interrupted, the process moves from the **Running** state back to the **Not Running** state.
5. The dispatcher then chooses another process from the **Not Running** state and moves it to the **Running** state.

The Creation and Termination of Processes

Before we delve deeper into improving the basic two-state model, it's important to first explore how processes begin and end. No matter what framework we use to describe process behavior, every process has a

clearly defined lifespan—it starts when it’s created and concludes when it’s terminated. These two events mark the boundaries of its existence.

Process Creation

Process Creation Method	Description
New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive log-on	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

1. New Batch Job

- Batch jobs are collections of commands or programs submitted to the OS for sequential execution.
- Historically, these jobs were stored on media like punch cards, tapes, or disks.

- The OS reads the job control commands and creates a process to execute them.
- This is common in large-scale systems where multiple tasks need to be processed automatically without user interaction.
- Example: In mainframe systems, financial report generation is done as a batch job.

2. Interactive Log-on

- Occurs when a user logs into the system from a terminal or graphical interface.
- The OS creates a new process (often called a user session or shell) to handle user interaction.
- This process manages tasks like command execution, running applications, and user-specific settings.
- Example: When logging into Linux, the OS spawns a shell process like `bash`.

3. Created by OS to Provide a Service

- The OS sometimes needs to create processes that serve user programs.
- These processes handle auxiliary or background tasks without requiring the user to wait.
- Such processes improve efficiency by offloading specific responsibilities.

- Example: A print spooler process manages printing requests so that the main program does not block while printing.
- Other examples include daemons in UNIX (e.g., `cron`, `sshd`) that provide continuous services.

4. Spawned by Existing Process

- A running process can create (spawn) one or more child processes to share its workload.
- This technique supports modularity, allowing different tasks to run independently.
- It also enables parallelism, where multiple processes execute simultaneously, improving performance.
- Example: A web server process (like Apache) spawns multiple worker processes to handle incoming client requests concurrently.
- In UNIX systems, this is often done using the `fork()` system call.

Process Termination

We summarize the reasons for process termination as follow

Normal completion	The process executes an OS service call to indicate that it has completed running.
Time limit exceeded	The process has run longer than the specified total time limit. There are a number of possibilities for the type of time that is measured. These include total elapsed time ("wall clock time"), amount of time spent executing, and, in the case of an interactive process, the amount of time since the user last provided any input.
Memory unavailable	The process requires more memory than the system can provide.
Bounds violation	The process tries to access a memory location that it is not allowed to access.
Protection error	The process attempts to use a resource such as a file that it is not allowed to use, or it tries to use it in an improper fashion, such as writing to a read-only file.
Arithmetic error	The process tries a prohibited computation (such as division by zero) or tries to store numbers larger than the hardware can accommodate.
Time overrun	The process has waited longer than a specified maximum for a certain event to occur.
I/O failure	An error occurs during input or output, such as inability to find a file, failure to read or write after a specified maximum number of tries (when, for example, a defective area is encountered on a tape), or invalid operation (such as reading from the line printer).
Invalid instruction	The process attempts to execute a nonexistent instruction (often a result of branching into a data area and attempting to execute the data).
Privileged instruction	The process attempts to use an instruction reserved for the operating system.
Data misuse	A piece of data is of the wrong type or is not initialized.
Operator or OS intervention	For some reason, the operator or the operating system has terminated the process (e.g., if a deadlock exists).
Parent termination	When a parent terminates, the operating system may automatically terminate all of the offspring of that parent.
Parent request	A parent process typically has the authority to terminate any of its offspring.

Figure 3.6: *Reasons for Process Termination*

1. Normal Completion

A process normally terminates when it successfully completes its execution. It issues an operating system service call to indicate that it has finished, and resources such as memory and I/O devices are released.

2. Time Limit Exceeded

Every process is usually given a time limit. If the process exceeds its wall-clock time, CPU time, or input wait time, the OS terminates it to ensure fairness and efficiency in resource usage.

3. Memory Unavailable

If a process requests more memory than the system can allocate, the OS may terminate it. This prevents memory starvation for other processes.

4. Bounds Violation

Processes are only allowed to access their allocated memory space. If a process attempts to access memory outside this range (illegal addressing), the OS terminates it to protect data integrity.

5. Protection Error

A process may attempt to use resources incorrectly, such as writing to a read-only file or accessing a restricted device. Such actions are blocked, and the process may be terminated.

6. Arithmetic Error

Invalid mathematical operations (e.g., division by zero, overflow beyond hardware limits) cause the process to terminate since the result is undefined or cannot be handled.

7. Time Overrun

If a process is waiting for an event (e.g., I/O completion) longer than a specified maximum wait time, the system terminates it to avoid blocking resources indefinitely.

8. I/O Failure

When input/output errors occur, such as missing files, device malfunctions, or invalid I/O operations, the process is terminated since it cannot continue reliably.

9. Invalid Instruction

This occurs if a process tries to execute a non-existent machine instruction. Often this happens due to memory corruption or incorrect branching.

10. Privileged Instruction

Certain instructions are reserved for the OS kernel. If a user process attempts to execute such instructions, the OS terminates it to maintain system stability.

11. Data Misuse

Processes sometimes misuse data by using uninitialized variables or assigning incompatible data types. This leads to termination to prevent corruption.

12. Operator or OS Intervention

The system administrator or OS may terminate a process deliberately, for example, in case of a deadlock or performance issue.

13. Parent Termination

If a parent process terminates, the OS may terminate its child processes as well, to prevent orphan processes from consuming system resources.

14. Parent Request

A parent process typically has control over its child processes and can terminate them explicitly using system calls (e.g., kill in UNIX/Linux).

A Five-State Model

The two-state process model, which classifies processes as either running or not running, is inefficient because the not running state mixes both ready and blocked processes, forcing the dispatcher to scan the queue to find a process that is actually ready. To overcome this limitation, the five-state process model was introduced, which separates the not running state into ready and blocked (waiting) states and also adds two additional states:

- **Running:** The process currently using the CPU (only one at a time in a single-processor system).
- **Ready:** A process prepared to execute as soon as the CPU becomes available.
- **Blocked/Waiting:** A process waiting for an event (e.g., I/O completion) before it can continue execution.
- **New:** A process that has been created but not yet admitted into main memory for execution. Its PCB (Process Control Block) is created, but program code and data remain in secondary storage.
- **Exit:** A process that has finished execution (normal completion, error, or forced termination). At this stage, the OS keeps temporary records for accounting, billing, or performance monitoring before fully deleting the process.

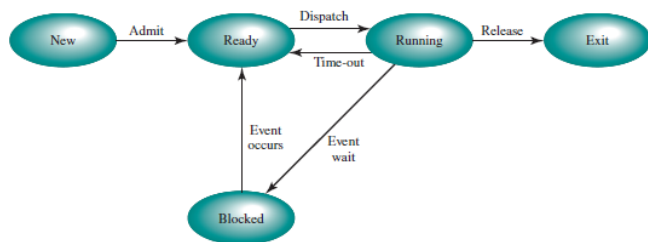


Figure 3.7: *Five-State Process Model*

Figure 3.7 indicates the types of events that lead to each state transition for a process; the possible transitions are as follows:

- **Null → New:** A process is created to begin executing a program. This creation may occur due to any of the reasons listed in Table 3.1.
- **New → Ready:** Once the system is able to handle another process, the OS moves it from the New state to the Ready state. This decision depends on limits such as the number of active processes or available virtual memory, ensuring that performance is not degraded.
- **Ready → Running:** The scheduler or dispatcher selects one of the processes in the Ready state to execute. This scheduling mechanism is discussed in more detail in Part Four.

- **Running** → **Exit**: A process in execution is terminated by the OS either because it has completed successfully or it has aborted. (Refer to Table 3.2 for details.)
- **Running** → **Ready**: The process may be moved back to Ready if:
 - It exceeds its maximum allowed execution time (time-slicing).
 - A higher-priority process becomes ready, causing preemption of the current process.
 - It voluntarily yields the processor, such as background tasks that perform periodic work.
- **Running** → **Blocked**: A process enters the Blocked state when it makes a request that requires waiting. Examples include:
 - Requesting a system service that is not immediately available.
 - Waiting for a resource (e.g., file, memory).
 - Waiting for I/O operations to finish.
 - Waiting for communication from another process (e.g., data or messages).
- **Blocked** → **Ready**: When the event that a process was waiting for occurs, it transitions back to the Ready state.
- **Ready** → **Exit**: Not shown in the diagram, but in some systems a parent process can terminate its child process at any time. Additionally, when a parent terminates, all its child processes may also be terminated.

- **Blocked** → **Exit**: Similar to the above, a blocked process can be terminated by its parent or due to the termination of its parent process.

Figure 3.8 shows the transition of each process among the states.

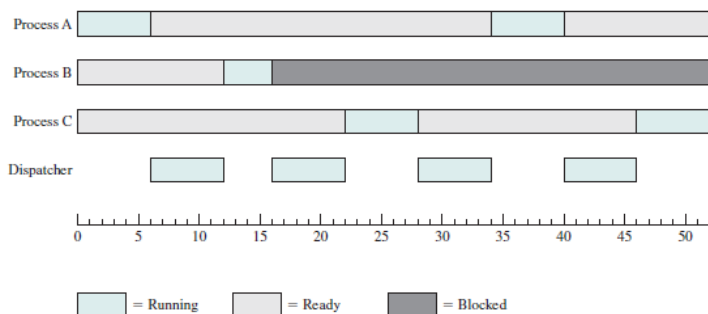


Figure 3.8: *trace processes states*

Figure 3.9a illustrates a basic queueing discipline using two primary queues:

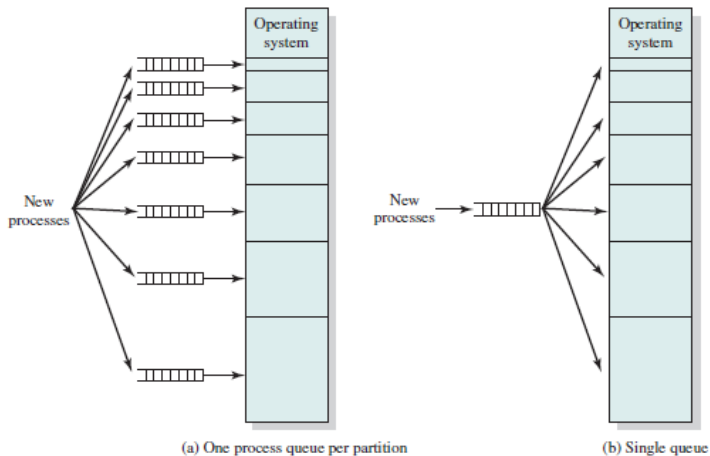


Figure 3.9: *process queuing discipline*

- **Ready Queue:** Holds processes that are ready to execute.
- **Blocked Queue:** Holds processes waiting for specific events (e.g., I/O).

When a process is admitted to the system, it is placed in the Ready queue. The operating system selects a process from this queue to run, typically using a first-in-first-out (FIFO) strategy if no priority scheme is applied. Once a process finishes its CPU burst, it may be:

- Terminated,
- Returned to the Ready queue (e.g., preempted),

- Moved to the Blocked queue (e.g., waiting for an event).

When an event occurs, the OS must scan the Blocked queue to identify processes waiting on that event. In large systems, this scanning becomes inefficient due to the high number of blocked processes.

Improved Model (Figure 3.9b):

- Use multiple **event-specific Blocked queues**, each associated with a distinct event.
- When an event occurs, all processes in the corresponding queue are moved to the Ready queue directly.

Additionally, if a **priority-based scheduling** scheme is used, it is beneficial to maintain multiple Ready queues—one for each priority level. This allows the OS to efficiently select the highest-priority process that has been waiting the longest.

Suspended Processes

In a uniprogramming system, the CPU often sits idle because input/output (I/O) operations are much slower than computation. Multiprogramming improves CPU utilization by allowing the processor to switch to another process when one is waiting for I/O. However, even with multiprogramming, the CPU may still remain idle if all processes in memory are waiting for I/O. To solve this, operating systems use **swapping** and introduce new **suspend states** into the process model.

Problem with Multiprogramming

Multiprogramming allows multiple processes to reside in main memory, but the processor is so much faster than I/O devices that it is common for all of them to be waiting for I/O simultaneously. When this happens, the processor still has no ready process to run and becomes idle. Thus, multiprogramming alone does not fully solve the problem of processor underutilization.

Expanding Main Memory

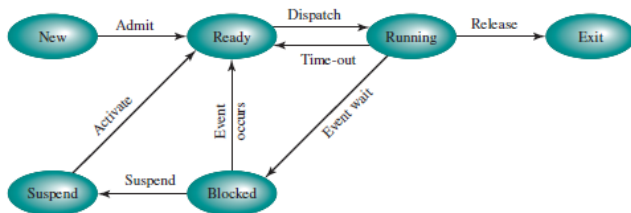
One possible solution is to increase main memory so that more processes can be loaded at once. However, this approach has two major drawbacks. First, expanding memory is costly. While the cost per byte has decreased over time, the total cost rises significantly as storage reaches gigabyte levels. Second, programs tend to grow larger as memory becomes more available. Instead of enabling more processes, additional memory often just supports larger processes, leaving the underlying problem unresolved.

Swapping as a Solution

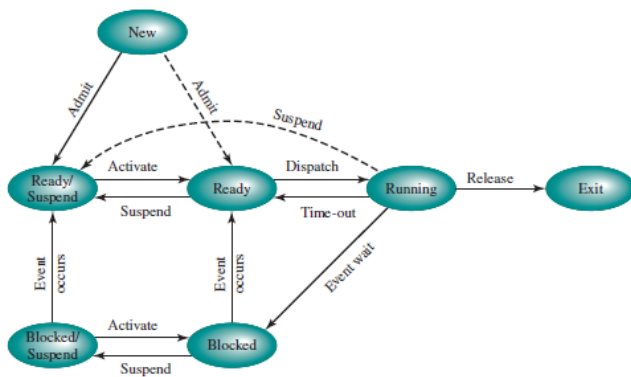
A better solution is **swapping**, where part or all of a process is moved from main memory to disk. If all processes in memory are blocked, the operating system swaps one of them to a **suspend queue** on disk, freeing space in main memory. The OS can then load a new process or bring back a previously suspended one. Although swapping itself requires I/O, disk operations are faster than many other devices such as tape or printers, so swapping typically enhances system performance by keeping the CPU busy.

The Suspend State

Swapping introduces a new state called **Suspend**. A suspended process has been removed from main memory and placed on disk as in figure 3.10a. The operating system then decides whether to load a new process or reload a suspended one. However, reloading a suspended process is not always useful because processes are usually suspended while blocked. Reloading a process that is still waiting for an event means it still cannot execute, which makes the decision more complex.



(a) With one Suspend state



(b) With two Suspend states

Figure 3.10: Process transition with suspend states

Two Dimensions of Process Status

To manage this complexity, process status must be described using two independent dimensions:

1. Whether the process is **blocked or not blocked** (waiting for an

event vs. ready to run).

2. Whether the process is **in main memory or swapped out**.

This creates four possible process states:

- **Ready:** In memory and ready for execution.
- **Blocked:** In memory but waiting for an event.
- **Blocked/Suspend:** On disk (swapped out) and waiting for an event.
- **Ready/Suspend:** On disk but ready to execute once brought into memory.

This expanded state model allows the operating system to make better decisions about process scheduling and memory management.

The state transition model shown in Figure 3.10b introduces new transitions involving the suspend states. Dashed lines in the figure indicate optional transitions that may or may not occur depending on system conditions. The key transitions are described as follows:

1. *Blocked* $\rightarrow$ *Blocked/Suspend*

If no ready processes are available, the operating system may swap out a blocked process to free memory for another process that is not blocked. This transition can also occur even when ready processes exist. For example, if the OS determines that the running or a ready process requires additional memory for acceptable performance, a blocked process may be suspended.

2. *Blocked/Suspend* $\rightarrow$ *Ready/Suspend*

When the event that a suspended blocked process has been waiting for occurs, the process is no longer blocked. It is then moved to the Ready/Suspend state. This requires that the OS maintain accessible state information for processes that are suspended.

3. *Ready/Suspend* $\rightarrow$ *Ready*

If there are no ready processes currently in main memory, the OS must bring in a process from the Ready/Suspend state to continue execution. Additionally, if a Ready/Suspend process has higher priority than any process in the Ready state, the OS may decide to load it immediately, prioritizing responsiveness over minimizing swapping operations.

4. *Ready* $\rightarrow$ *Ready/Suspend*

Normally, the OS prefers to suspend blocked processes rather than ready ones, since blocked processes cannot execute while occupying memory. However, in some cases it may be necessary to suspend a ready process. This may happen if it is the only way to free a sufficiently large block of memory. The OS may also suspend a lower-priority ready process instead of a higher-priority blocked process if it expects that the blocked process will soon become ready.

Beyond the core suspend-related transitions, there are additional transitions worth considering. One important case involves the handling of newly created processes.

- **New $\rightarrow$ Ready/Suspend and New $\rightarrow$ Ready** When a new process is created, the operating system can place it into either the **Ready** state or the **Ready/Suspend** state. In both situations, the OS must

create a process control block and allocate an address space for the process.

- **Blocked/Suspend → Blocked** At first glance, bringing a process from the *Blocked/Suspend* state back into the *Blocked* state might seem inefficient. If a process is blocked and not in main memory, loading it could appear pointless. However, consider a scenario in which a process terminates and releases memory. If there exists a process in the *Blocked/Suspend* queue with a higher priority than any process in the *Ready/Suspend* queue, and the operating system anticipates that the blocking event will occur soon, it is reasonable to move the blocked process into main memory. This ensures that it can resume execution immediately once unblocked.
- **Running → Ready/Suspend** Typically, when a process exhausts its allocated time slice, it is moved from the *Running* state to the *Ready* state. However, in some cases, the operating system may choose to move it directly to the *Ready/Suspend* state. For instance, if a higher-priority process in the *Blocked/Suspend* queue becomes unblocked, the OS may preempt the running process and suspend it to free up memory for the higher-priority task.
- **Any State → Exit** Most often, a process moves to the *Exit* state while running, either because it has completed execution or due to a fatal error. Nevertheless, some operating systems allow a process to be terminated externally, either by its parent process or when the parent itself terminates. In such cases, a process can transition to the *Exit* state from *any* other state.

The choice of transition depends on the system's design philosophy. One approach is to perform these tasks early, so that the OS maintains a large pool of processes that are not blocked. However, this can quickly lead to a shortage of available main memory, making the **New** → **Ready/Suspend** transition necessary.

On the other hand, the OS could follow a just-in-time approach, delaying process creation until it is truly required. This reduces OS overhead and shifts process creation duties to moments when the system is already heavily occupied with blocked processes. this table lists some reasons for the suspension of a process.

Table 3.3 Reasons for Process Suspension

Swapping	The OS needs to release sufficient main memory to bring in a process that is ready to execute.
Other OS reason	The OS may suspend a background or utility process or a process that is suspected of causing a problem.
Interactive user request	A user may wish to suspend execution of a program for purposes of debugging or in connection with the use of a resource.
Timing	A process may be executed periodically (e.g., an accounting or system monitoring process) and may be suspended while waiting for the next time interval.
Parent process request	A parent process may wish to suspend execution of a descendent to examine or modify the suspended process, or to coordinate the activity of various descendants.

Figure 3.11: *reasons of suspension*

3.3 Process Description

To manage processes and resources efficiently, the operating system (OS) must maintain up-to-date information about the status of each process and resource. This information is organized in the form of **tables**, which the OS constructs and maintains. Fundamentally, most

operating systems use four main categories of tables: **memory, I/O, file, and process tables.**

Memory Tables

Memory tables track the allocation and use of both **main (real) memory** and **secondary (virtual) memory**. Key points include:

- Main memory is partially reserved for OS use, with the remainder available for processes.
- Secondary memory may store processes using **virtual memory** or **swapping**.
- Memory tables typically contain:
 - Allocation of main and secondary memory to processes.
 - Protection attributes (e.g., which processes can access shared memory).
 - Information for managing virtual memory.

I/O Tables

I/O tables help the OS manage **input/output devices and channels**. Important points:

- At any moment, an I/O device may be free or assigned to a specific process.
- The OS tracks the **status of ongoing I/O operations** and the memory locations used for I/O transfers.

File Tables

File tables provide information about **files and storage**. They include:

- Existence and location of files on secondary storage.
- Current status and attributes of files.
- Depending on the OS, file management may be handled mostly by a **file management system** or directly by the OS.

Process Tables

Process tables store information required to **manage processes**. Key aspects:

- They are central because memory, I/O, and files are all resources used by processes.
- Process tables often **cross-reference memory, I/O, and file tables** to track resource usage.
- Tables must be accessible by the OS and are themselves subject to memory management.

Table Initialization and Configuration

- The OS needs knowledge of the **basic environment** to create tables:
 - Amount of main memory.
 - Available I/O devices and identifiers.
 - Initial configuration data.

- These data are provided externally, either by human input or automated configuration tools, during OS initialization.

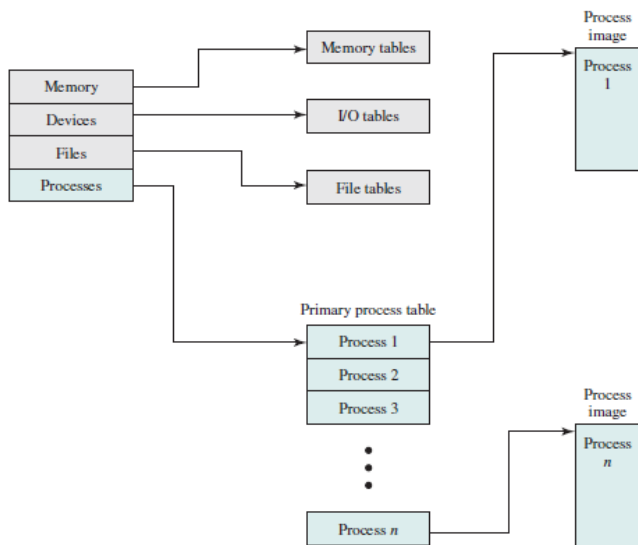


Figure 3.12: General Structure of Operating System Control Tables

3.4 Process Control

Modes of Execution

Modern processors support at least **two execution modes**: one for user programs and one for operating system (OS) functions. These modes are

necessary to protect the OS and critical data from interference by user programs.

User Mode vs. Kernel Mode

- **User Mode:**

- Less-privileged mode, used for running user programs.
- Cannot execute sensitive instructions such as I/O operations or memory management.

- **Kernel (System) Mode:**

- More-privileged mode, used by the OS.
- Can execute all instructions and access all memory, registers, and control structures.
- Protects critical OS components like process control blocks and memory tables.

Purpose of Two Modes

- Protect the OS from accidental or malicious interference by user programs.
- Restrict sensitive operations to the kernel while allowing safe execution of user applications.

Mode Switching

- The processor knows its current mode via a **bit in the processor status word (PSW)** or status register.

- Mode changes occur in response to events such as:
 - System calls made by user programs.
 - Hardware interrupts requiring OS intervention.
- Example: Intel Itanium (IA-64 architecture)
 - Processor status register (psr) includes a 2-bit **current privilege level (CPL)** field.
 - Level 0 = most privileged (kernel), Level 3 = least privileged (user).
 - On an interrupt or system call:
 - * CPL is automatically set to level 0.
 - * Upon return, the IRT (interrupt return) instruction restores the previous mode.
 - System calls transfer control from user mode to kernel mode by storing arguments in a predefined area and executing a special instruction.

Process Creation

After deciding to create a new process, the operating system (OS) follows a series of steps to properly initialize and manage the process.

1. Assign Process Identifier (PID)

- The OS assigns a **unique process identifier** to the new process.

- A new entry is added to the **primary process table**, which maintains one entry per process.

2. Allocate Space for the Process

- Allocate memory for the **process image**, including:
 - Private user address space (programs and data).
 - User stack.
- Allocation values can be **default** or **user-specified**.
- If spawned by a parent process, the parent can provide memory requirements.
- Set up any **shared address spaces** if required.
- Allocate space for the **process control block (PCB)**.

3. Initialize the Process Control Block (PCB)

- **Process identification:** Set process ID and parent process ID.
- **Processor state:** Initialize most entries to zero, except:
 - Program counter → program entry point.
 - System stack pointers → process stack boundaries.
- **Process control information:**
 - Initialize state (typically **Ready** or **Ready/Suspend**).
 - Set priority (default lowest unless specified).
 - Initially, process owns no resources unless requested or inherited from parent.

4. Set Linkages

- Insert the new process into the appropriate **scheduling queue** (e.g., Ready or Ready/Suspend list) if queues are maintained as linked lists.

5. Create or Expand Other Data Structures

- Maintain additional structures such as **accounting files** for billing, auditing, or performance assessment.

Process Switching

Process switching involves interrupting the currently running process and assigning the CPU to another process. Several design considerations arise:

- Determining the **events that trigger a process switch**.
- Distinguishing between **mode switching** (user/kernel) and **process switching**.
- Updating the **data structures** required to implement the switch.

Events Triggering Process Switching

A process switch can occur whenever the OS gains control of the CPU. Key events include:

System Interrupts

Interrupts are events external to the running process, while traps are exceptions or errors generated by the process itself.

- **Clock interrupt:** Checks whether the running process has used its maximum allowed time slice. If so, the process moves to the Ready state, and another process is dispatched.
- **I/O interrupt:** When an I/O operation completes, the OS moves waiting blocked processes to Ready (and Blocked/Suspend to Ready/Suspend). The OS then decides whether to resume the current process or switch to a higher-priority Ready process.
- **Memory fault:** When a process accesses a virtual memory address not in main memory, the OS loads the required block from secondary memory, places the process in a Blocked state, and switches to another process. Once the block is loaded, the process moves to Ready.

Traps

Generated by the process due to errors or exceptions:

- **Fatal trap:** The process is terminated and moved to the Exit state, triggering a process switch.
- **Non-fatal trap:** The OS may attempt recovery, notify the user, or decide whether to switch processes or resume the current one.

System (Supervisor) Calls

When a process requests OS services (e.g., file operations), the OS may place the process in a Blocked state while the service is executed.

Mode Switching

Mode switching refers to changing the processor from **user mode** to **kernel mode** to execute privileged instructions, usually in response to an interrupt or system call:

1. Set the program counter to the start of the interrupt handler.
2. Switch from user mode to kernel mode.
3. Save the context of the interrupted process in its Process Control Block (PCB), including the program counter, processor registers, and stack information.
4. Execute the interrupt handler to perform tasks such as resetting interrupt flags, acknowledging I/O devices, or handling clock events.

Mode switching may occur **without a process switch**, requiring only context saving and restoration.

Full Process Switching

When a process switch occurs, the OS must perform additional steps:

1. Save the processor context (program counter, registers).
2. Update the PCB of the currently running process, including its new state (Ready, Blocked, Ready/Suspend, or Exit), reason for leaving Running, and accounting information.
3. Move the PCB to the appropriate scheduling queue.
4. Select the next process to execute.

5. Update the PCB of the selected process, changing its state to Running.
6. Update memory management data structures if required (e.g., for address translation).
7. Restore the processor context for the selected process.

3.5 *Exercise*

1. What is an instruction trace?
2. What is process spawning?
3. Why does Figure 3.9b have two blocked states?
4. List four characteristics of a suspended process.
5. For what types of entities does the OS maintain tables of information for management purposes?
6. What are the elements of a process image?
7. Why are two modes (user and kernel) needed?
8. What are the steps performed by an OS to create a new process?
9. What is the difference between an interrupt and a trap?
10. Give three examples of an interrupt.
11. What is the difference between a mode switch and a process switch?

True/False Questions

1. T F A computer platform consists of a collection of hardware resources, such as the processor, main memory, I/O modules, timers, and disk drives.
2. T F For efficiency, applications should be written directly for a given hardware platform.
3. T F A design change in the structure or semantics of the process control block could affect a number of modules in the OS.
4. T F The process control block is the key tool that enables the OS to support multiple processes and to provide for multiprocessing.
5. T F It is not the responsibility of the operating system to control the execution of processes.
6. T F The first step in designing an OS to control processes is to describe the behavior that we would like the processes to exhibit.
7. T F The OS may create a process on behalf of an application.
8. T F Swapping is not an I/O operation so it will not enhance performance.
9. T F If a system does not employ virtual memory, each process to be executed must be fully loaded into main memory.

10. T F A process that is not in main memory is immediately available for execution, regardless of whether or not it is awaiting an event.
11. T F The OS may suspend a process if it detects or suspects a problem.
12. T F All processor designs include a register or set of registers, often known as the program status word, which contains status information.
13. T F The process control block is the least important data structure in an OS.
14. T F A process switch may occur any time that the OS has gained control from the currently running process.
15. T F The principal function of the OS is to create, manage, and terminate processes.

Multiple Choice Questions

1. The processor itself provides only limited support for multiprogramming, and is needed to manage the sharing of the processor and other resources by multiple applications at the same time.
 - a) memory
 - b) data
 - c) software

d) hardware

2. "The process was placed in a suspended state by an agent; either itself, a parent process, or the OS, for the purpose of preventing its execution," is a characteristic of a process.

- a) blocked
- b) suspended
- c) ready
- d) swapped

3. A(n) is a unit of activity characterized by the execution of a sequence of instructions, a current state, and an associated set of system resources.

- a) identifier
- b) process
- c) state
- d) kernel

4. We can characterize the behavior of an individual process by listing the sequence of instructions, referred to as a, that executes for that process.

- a) state
- b) trace
- c) process block

- d) priority
5. It is the principal responsibility of the to control the execution of processes.
- a) OS
 - b) process control block
 - c) memory
 - d) dispatcher
6. involves moving part or all of a process from main memory to disk.
- a) Swapping
 - b) Relocating
 - c) Suspending
 - d) Blocking
7. When a process is in the state it is in secondary memory but is available for execution as soon as it is loaded into main memory.
- a) Blocked
 - b) Blocked/Suspend
 - c) Ready
 - d) Ready/Suspend
8. A process is in the state when it is in main memory and awaiting an event.

- a) Blocked
 - b) Blocked/Suspend
 - c) Ready/Suspend
 - d) Ready
9. The OS must maintain..... tables to manage processes.
- a) process
 - b) I/O
 - c) memory
 - d) file
10. The collection of program, data, stack, and attributes is referred to as the
- a) process structure
 - b) process control block
 - c) process location
 - d) process image
11. The is the less-privileged mode.
- a) user mode
 - b) kernel mode
 - c) system mode
 - d) control mode

Chapter 4

Threads

4.1 Introduction and Motivation

- A **thread** is the smallest unit of CPU utilization.
- Consists of: thread ID, program counter, register set, and stack.
- Threads share code, data, and resources of the process.
- **Motivation:**
 - Modern applications are multithreaded (e.g., browsers, servers).
 - Thread creation is lightweight compared to process creation.
 - Improves performance on multicore systems and simplifies design.
- **Benefits:**
 - Responsiveness – UI stays active while other tasks wait.
 - Resource sharing – threads share memory and resources.

- Economy – less overhead than process creation.
- Scalability – effective use of multiprocessors.

4.2 *Multicore Programming*

- Challenges: dividing tasks, balancing load, avoiding data conflicts, debugging.
- **Concurrency** – multiple tasks making progress (can be on single core).
- **Parallelism** – true simultaneous execution on multiple cores.
- **Types of Parallelism:**
 - Data parallelism – same operation applied to subsets of data.
 - Task parallelism – different tasks executed on different cores.
- **Amdahl's Law:**
 - Speedup is limited by serial portion of a program.
 - If 25% is serial, max speedup is 4x regardless of core count.



Concurrency vs. Parallelism

■ Concurrent execution on single-core system:



■ Parallelism on a multi-core system:

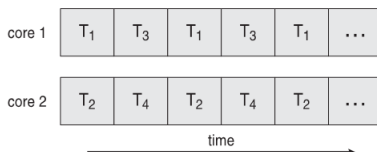


Figure 4.1: concurrency Vs. parallelism

4.3 Multi threading Models

• User Threads vs. Kernel Threads:

- User threads – managed by user libraries (fast, limited).
- Kernel threads – managed by the OS kernel (slower, powerful).

• Models:

1. Many-to-One – many user threads mapped to one kernel

- thread. Blocking one blocks all, no parallelism. Example: Solaris Green Threads.
2. One-to-One – each user thread maps to one kernel thread. True concurrency, higher overhead. Examples: Windows, Linux, Solaris 9+.
 3. Many-to-Many – many user threads mapped to many kernel threads. Balances efficiency and concurrency. Example: Solaris (old versions).
 4. Two-Level Model – variation of Many-to-Many, allows binding user thread to kernel thread. Examples: HP-UX, Solaris 8.

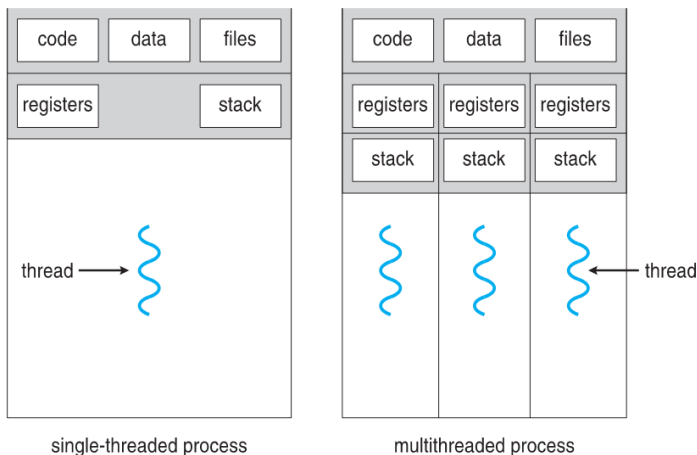


Figure 4.2: *single and multi threads*

4.4 Thread Libraries

- Provide API for creating, synchronizing, and managing threads.
- Implemented in user space or kernel space.
- **Popular Libraries:**
 - **Pthreads:** POSIX standard, common in UNIX/Linux/macOS.
 - **Windows Threads:** Part of Windows API, one-to-one kernel threads.
 - **Java Threads:** Managed by JVM, map to native OS threads.

4.5 *Implicit Threading*

- Motivation: Writing correct explicit threads is complex.
- Compiler/runtime manages threads instead of programmer.
- **Techniques:**
 1. Thread Pools – reuse worker threads, reduce overhead.
 2. OpenMP – directives for parallel programming in C/C++/Fortran.
 3. Grand Central Dispatch (GCD) – Apple technology, uses dispatch queues (serial or concurrent).
 4. Other methods – Microsoft TBB, Java concurrency utilities.

4.6 *Threading Issues*

- **Semantics of fork() and exec():**
 - fork may duplicate only calling thread or all threads.
 - exec replaces entire process (all threads).
- **Signal Handling:**
 - Signals notify processes of events in UNIX.
 - In multithreaded programs, signals may go to: the causing thread, all threads, or a dedicated handler thread.
- **Thread Cancellation:**

- Asynchronous – terminates immediately (unsafe).
 - Deferred – checked periodically, safer (default in Pthreads).
- **Thread-Local Storage (TLS):** Each thread has its own copy of data.
- **Scheduler Activations:**
 - Uses Lightweight Processes (LWPs) between user and kernel threads.
 - Kernel sends upcalls to maintain correct number of threads.

4.7 Operating System Examples

Windows

- Implements one-to-one mapping.
- Thread context: registers, user/kernel stacks, private storage.
- Data Structures:
 - ETHREAD – executive thread block.
 - KTHREAD – kernel thread block, scheduling info.
 - TEB – thread environment block, thread-local storage.

Linux

- Threads are called **tasks**.
- Created with `clone()` system call.

- `clone()` options:
 - `CLONE_VM` – share memory space.
 - `CLONE_FS` – share file system info.
 - `CLONE_FILES` – share open files.
 - `CLONE_SIGHAND` – share signal handlers.
- Implemented with `task_struct`.

4.8 Exercises

Open-Ended Questions

1. Explain the main differences between a process and a thread. Why are threads considered lightweight?
2. Describe at least three benefits of multithreading in modern operating systems.
3. What are the major challenges of programming for multicore systems? Provide examples.
4. Differentiate between concurrency and parallelism with suitable examples.
5. Explain Amdahl's Law and its impact on performance gains in multicore programming.
6. Compare the Many-to-One, One-to-One, and Many-to-Many threading models. Mention their advantages and disadvantages.

7. What are the main differences between Pthreads, Windows threads, and Java threads?
8. What is implicit threading, and why has it become more important in modern systems?
9. Explain the role of thread-local storage (TLS). How is it different from normal local variables?
10. Compare the implementation of threads in Windows and Linux operating systems.

Multiple Choice Questions (MCQs)

1. A thread consists of which of the following?
 - A) Thread ID, program counter, register set, stack
 - B) Code, data, stack, program counter
 - C) Process ID, code, heap, register set
 - D) Thread ID, code, heap, data
2. Which of the following is a key advantage of multithreading?
 - A) Increased memory usage
 - B) Faster context switching between processes
 - C) Improved responsiveness and resource sharing
 - D) No synchronization needed
3. Concurrency refers to:
 - A) Running multiple tasks simultaneously on multiple cores

- B) Making progress on multiple tasks, possibly with one core
 - C) Splitting data across processors for computation
 - D) Synchronizing multiple threads in parallel
4. According to Amdahl's Law, the speedup of a program is limited by:
- A) The number of cores
 - B) The size of the data
 - C) The serial portion of the program
 - D) The thread scheduling algorithm
5. Which threading model maps multiple user threads to a single kernel thread?
- A) One-to-One
 - B) Many-to-One
 - C) Many-to-Many
 - D) Two-Level Model
6. In which operating systems is the One-to-One model primarily used?
- A) Windows and Linux
 - B) Solaris Green Threads
 - C) HP-UX and Solaris 8
 - D) Older versions of Solaris
7. Which thread library is a POSIX standard commonly used in UNIX/Linux?

- A) Windows API Threads
- B) Java Threads
- C) Pthreads
- D) OpenMP

8. Thread pools are beneficial because:
- A) They eliminate the need for threads
 - B) They reuse existing threads, reducing creation overhead
 - C) They prevent parallelism in applications
 - D) They are only supported in Linux
9. Which of the following is **NOT** a threading issue?
- A) Signal handling
 - B) Thread cancellation
 - C) Concurrency vs. Parallelism
 - D) Scheduler activations
10. In Linux, the `clone()` system call can be used to:
- A) Replace the current process with a new one
 - B) Create a new kernel thread without sharing any resources
 - C) Create tasks that selectively share memory, files, and signals
 - D) Cancel a thread immediately

Chapter 5

Memory Management

5.1 Memory Management Requirements

When exploring the different mechanisms and policies related to memory management, it is essential to consider the key requirements that such a system must satisfy. These requirements include:

- **Relocation** – the ability to move programs in memory while they are running.
- **Protection** – ensuring that one process cannot access or modify another process's memory.
- **Sharing** – providing mechanisms for processes to share memory safely when needed.
- **Logical organization** – structuring programs into modules to support protection and modular execution.
- **Physical organization** – handling the mapping between main memory and secondary storage efficiently.

Table 5.1: *Definitions of Memory terms*

Frame	A fixed-length block of main memory.
Page	A fixed-length block of data that resides in secondary memory (such as a disk). A page of data may temporarily be copied into a frame of main memory.
Segment	A variable-length block of data that resides in secondary memory. An entire segment may temporarily be copied into an available region of main memory (segmentation) or the segment may be divided into pages, which can be individually copied into main memory (combined segmentation and paging).

Relocation

In a multiprogramming environment, the main memory is shared among multiple processes. It is not possible to predict in advance which programs will reside in memory during execution. To improve processor utilization, active processes may be swapped in and out of memory. When a process is swapped back into main memory, it cannot always return to its original location; therefore, processes must be capable of relocation. These facts raise some technical concerns related to addressing, as illustrated in Figure

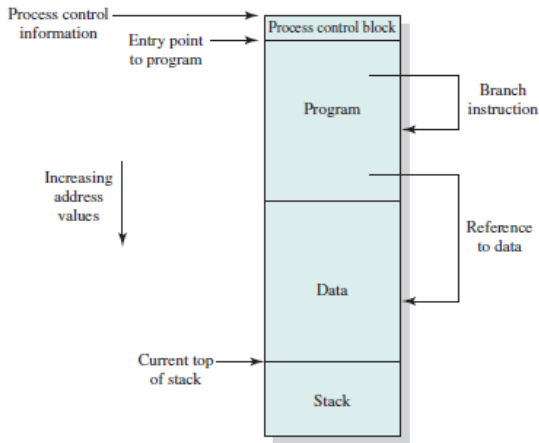


Figure 5.1: Addressing Requirements for a Process

This situation introduces technical concerns related to **addressing**, since memory references within the program must be translated to reflect the process's current location in main memory.

Protection

Each process must be protected from accidental or intentional interference by other processes. This means that a program in one process should not be able to read from or write to the memory space of another process without proper authorization.

Satisfying the protection requirement becomes more challenging when combined with **relocation**. Since processes are loaded into unpredictable memory locations, absolute addresses cannot be validated

at compile time. Furthermore, many programming languages allow dynamic address calculations at run time (e.g., array indexing or pointer arithmetic). Therefore, all memory references must be checked at execution time to ensure they refer only to the allocated space of the process. Mechanisms that enable relocation also help enforce protection.

In general, user processes cannot access the operating system's code or data, nor can they branch into or access the data areas of other processes. If such attempts occur, the processor must detect and abort them during execution.

Importantly, memory protection must be enforced by **hardware**, not solely by the operating system. The OS cannot anticipate every possible memory reference, and pre-screening programs for violations would be impractical. Thus, the processor must include the capability to verify and restrict memory references at run time.

Sharing

A protection mechanism must also be flexible enough to permit multiple processes to access the same region of main memory when needed. For instance, if several processes run the same program, it is more efficient to let them share a single copy of the program instead of creating separate copies. Similarly, cooperating processes may require shared access to common data structures.

Therefore, the memory management system must enable controlled sharing of memory areas while still maintaining essential protection. Interestingly, the same mechanisms that support relocation also provide support for sharing.

Logical Organization

In most computer systems, main memory is structured as a **linear address space**, represented as a sequence of bytes or words. Secondary storage is also organized in a similar linear fashion at the physical level. While this design corresponds to the underlying hardware, it does not reflect how programs are typically constructed.

Programs are usually divided into **modules**, where some modules are *read-only* or *execute-only*, while others contain modifiable data. If the operating system and hardware can handle programs in terms of modules, several benefits can be achieved:

- **Independent development:** Modules can be written and compiled separately, with references between them resolved dynamically at run time.
- **Flexible protection:** Different modules can be assigned varying protection levels (e.g., read-only, execute-only) with minimal overhead.
- **Module-level sharing:** Modules can be shared among processes. This approach aligns with the user's logical view of the program, making it straightforward for users to specify which modules should be shared.

Physical Organization

Main memory provides high-speed access but is more expensive and volatile, meaning its contents are lost when power is removed. Secondary memory, in contrast, is slower and cheaper, but non-volatile, making it suitable for long-term storage of programs and data.

In this two-level organization, main memory holds the currently executing programs and active data, while secondary memory provides permanent storage for all programs and datasets. Managing the flow of information between these two memory levels is a major system concern. Assigning this responsibility directly to programmers is both inefficient and undesirable.

Overlaying

One traditional method to cope with the limited capacity of main memory is known as **overlaying**. In this approach, a large program is divided into smaller modules, and multiple modules are mapped onto the same memory region. A controlling routine, often called the *overlay manager*, dynamically loads and replaces modules as they are needed during execution.

For example, consider a compiler with separate phases such as:

- Lexical analysis
- Parsing
- Optimization
- Code generation

Since only one phase is active at a time, overlaying allows each phase to occupy the same portion of memory, with the active module being swapped in as required.

While overlaying reduces main memory requirements, it also imposes significant burdens on programmers:

- Programs must be carefully structured and divided into modules.
- The programmer must anticipate execution flow and manually organize memory usage.
- Debugging and program maintenance become more difficult.

5.2 *Memory Partitioning*

The core function of memory management is to load processes into main memory so they can be executed by the processor. In nearly all modern multiprogramming systems, this is achieved through the concept of virtual memory, which relies on two fundamental techniques: segmentation and paging.

Before exploring virtual memory, it is helpful to first review simpler methods that do not make use of it. One such method is partitioning, which was once applied in older operating systems but is no longer in common use. The other two methods—simple paging and simple segmentation—are not employed on their own in current systems. Nevertheless, studying them separately provides useful background and helps in understanding how virtual memory works.

Technique	Description	Strengths	Weaknesses
Fixed Partitioning	Main memory is divided into a number of static partitions at system generation time. A process may be loaded into a partition of equal or greater size.	Simple to implement; little operating system overhead.	Inefficient use of memory due to internal fragmentation; maximum number of active processes is fixed.
Dynamic Partitioning	Partitions are created dynamically, so each process is loaded into a partition of exactly the same size as that process.	No internal fragmentation; more efficient use of main memory.	Inefficient use of processor due to the need for compaction to counter external fragmentation.
Simple Paging	Main memory is divided into a number of equal-size frames. Each process is divided into a number of equal-size pages of the same length as frames. A process is loaded by loading all of its pages into available, not necessarily contiguous, frames.	No external fragmentation.	A small amount of internal fragmentation.
Simple Segmentation	Each process is divided into a number of segments. A process is loaded by loading all of its segments into dynamic partitions that need not be contiguous.	No internal fragmentation; improved memory utilization and reduced overhead compared to dynamic partitioning.	External fragmentation.
Virtual Memory Paging	As with simple paging, except that it is not necessary to load all of the pages of a process. Nonresident pages that are needed are automatically brought in later.	No external fragmentation; higher degree of multiprogramming; large virtual address space.	Overhead of complex memory management.
Virtual Memory Segmentation	As with simple segmentation, except that it is not necessary to load all of the segments of a process. Nonresident segments that are needed are automatically brought in later.	No internal fragmentation, higher degree of multiprogramming; large virtual address space; protection and sharing support.	Overhead of complex memory management.

Figure 5.2: *Memory Management Techniques*

Fixed Partitioning

In most memory management approaches, a designated portion of main memory is reserved for the operating system, while the remaining space is allocated for user processes. The most straightforward method of managing this available memory is to divide it into partitions with fixed, unchanging boundaries.

Partition size

On Figure 5.2 illustrates two possible approaches to fixed partitioning. One approach is to divide memory into equal-sized partitions. In this scheme, any process that is smaller than or equal to the partition size can be placed in any free partition. If all partitions are occupied and no process is actively running or ready to run, the operating system may swap out a process from one partition and load another process into that space, ensuring the processor remains busy.

Despite its simplicity, equal-size fixed partitioning has two major drawbacks:

- **Programs larger than a partition:** A process may be too large to fit into a single partition. In such cases, the programmer must employ *overlays*, structuring the program so that only part of it resides in memory at any time. When another module is required, it is loaded into the program's partition, replacing the portion currently present.
- **Inefficient memory utilization:** Regardless of size, each process occupies an entire partition. For example, a program smaller than 2 Mbytes still consumes a full 8 Mbyte partition. This wasted space within a partition, caused by the mismatch between program size and partition size, is known as **internal fragmentation**.

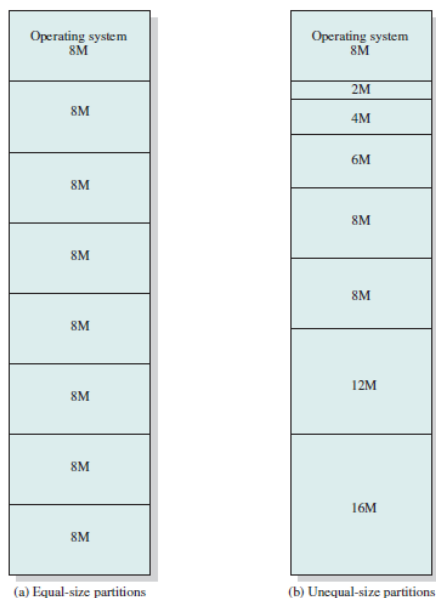


Figure 5.3: *fixed partitioning example*

The drawbacks of equal-size partitions can be reduced, though not completely eliminated, by using unequal-size partitions (Figure 5.3b). With this approach, larger programs—up to 16 Mbytes in this example—can be loaded without requiring overlays, while smaller partitions of less than 8 Mbytes make it possible to fit smaller programs more efficiently and reduce internal fragmentation.

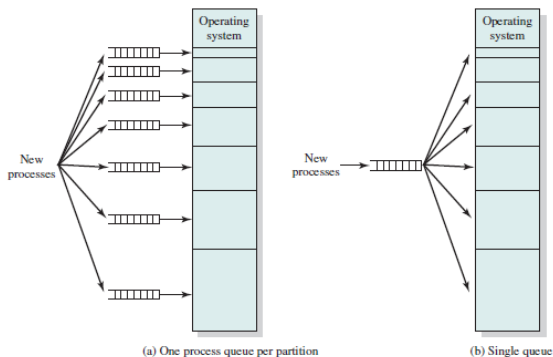


Figure 5.4: *fixed partitioning example*

Placement Algorithm

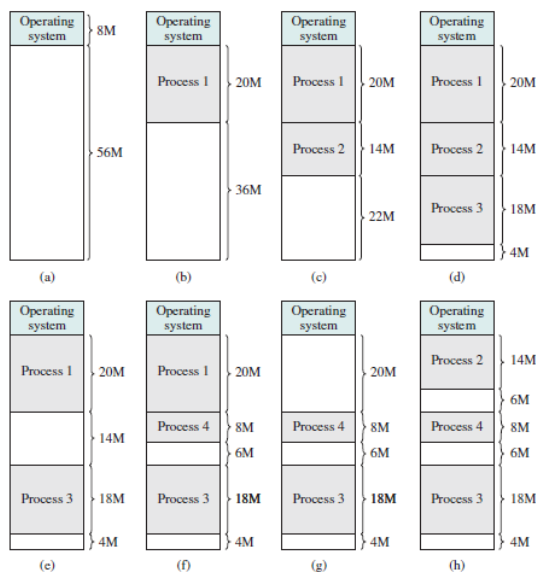
When using equal-size partitions, placing processes in memory is straightforward. A process can be loaded into any available partition, since all partitions are identical in size. If every partition is already occupied by processes that are not ready to execute, one of them must be swapped out to make space for the new process. The decision about which process to remove is a scheduling matter, discussed later in Part Four.

For unequal-size partitions, there are two possible strategies for placement. The simplest is to load each process into the smallest partition in which it fits. Under this method, a separate scheduling queue is maintained for each partition to store swapped-out processes intended for that partition (see Figure 5.4a). This approach minimizes wasted space inside partitions, thereby reducing internal fragmentation.

However, while this method is efficient from the perspective of each individual partition, it does not necessarily yield the most efficient results for the system as a whole. In Figure 5.3b, for example, consider a case in which there are no processes with a size between 12 and 16M at a certain point in time. In that case, the 16M partition will remain unused, even though some smaller process could have been assigned to it. Thus, a preferable approach would be to employ a single queue for all processes (see Figure 5.4b).

5.3 *Dynamic Partitioning*

To overcome some of the difficulties with fixed partitioning, an approach known as **dynamic partitioning**. Dynamic partitioning is a memory management scheme where partitions are not fixed in size or number. Instead, when a process is brought into main memory, it is allocated exactly the amount of memory it requires. This approach was notably used in IBM's OS/MVT (Multiprogramming with a Variable Number of Tasks).

Figure 5.5: *Dynamic partitioning example*

For example, assume we have **64 MB of main memory**. Initially, the memory contains only the operating system. As new processes are loaded, they are placed one after another, each occupying just the required memory space. This avoids the waste caused by unused space inside partitions (internal fragmentation), but introduces new problems.

Holes and External Fragmentation

As processes are swapped in and out of memory, small unused gaps begin to appear between allocated partitions. These unused gaps are

called **holes**.

Suppose process **P₂** is swapped out to make room for another process (say **P₄**). If **P₄** is smaller than **P₂**, a **hole** is left in memory. Over time, as processes of different sizes come and go, memory becomes increasingly divided into many scattered small holes.

This situation is known as **external fragmentation** because the free memory space is fragmented outside the allocated partitions.

Example (64 MB memory scenario):

- OS occupies the lower part of memory.
- Processes P₁, P₂, and P₃ are loaded sequentially.
- Later, P₂ is swapped out, creating a large hole.
- P₄ is loaded in the hole, but since P₄ is smaller than P₂, a leftover hole remains.
- As processes continue to be swapped, many small unusable holes accumulate.

Even though the total free memory may be large enough to hold a new process, the scattered holes prevent it from being allocated in one contiguous block.

Compaction: A Solution to External Fragmentation

To overcome external fragmentation, the operating system may perform **compaction**. Compaction involves shifting processes in memory so that they become contiguous, pushing all free holes together into one

large free block. After compaction, memory utilization improves, since the OS can load a new process into the consolidated free space.

Example (from Figure 5.5h case): After multiple swaps, suppose memory contains several small holes totaling 16 MB, scattered across different regions. By performing compaction, the OS moves processes together and merges all the holes into a single 16 MB block. Now, a new process requiring 16 MB can be loaded successfully.

Drawbacks of Compaction

While compaction solves the problem of external fragmentation, it introduces new challenges. Shifting processes in memory is time-consuming and delays execution. It also wastes valuable CPU cycles since resources are spent rearranging memory rather than executing user programs. Moreover, compaction requires **dynamic relocation**, meaning it must be possible to move a program to a new memory location without invalidating its memory references.

Placement Algorithms

When dealing with dynamic partitioning, memory compaction is often necessary to reduce external fragmentation. However, since compaction is time-consuming and costly, operating system designers must carefully decide how to allocate processes to available memory blocks. The decision of which free block to assign a process to can significantly affect both memory utilization and system performance. To address this, several placement algorithms are commonly used, including **best-fit**, **first-fit**, and **next-fit**.

Best-Fit Algorithm

The best-fit algorithm selects the free block that is closest in size to the process being allocated. For example, if a process requires 10 MB of memory and there are holes of 12 MB, 20 MB, and 50 MB, the operating system will place the process in the 12 MB block, since it is the smallest block that can accommodate the request. The advantage of this method is that it preserves larger blocks for future allocations, which may be needed for bigger processes. However, the drawback is that it often leaves behind very small unusable holes, which can increase external fragmentation over time.

First-Fit Algorithm

The first-fit algorithm begins scanning memory from the start and places the process into the first free block that is large enough. Using the earlier example, if the memory contains a 12 MB hole followed by a 50 MB hole, the 10 MB process will be placed into the 12 MB hole without searching further. This method is faster than best-fit because it does not require scanning all of memory to find the closest match. The main disadvantage is that it tends to create many small holes at the beginning of memory, which may reduce efficiency over time.

Next-Fit Algorithm

The next-fit algorithm works similarly to first-fit but modifies the search strategy. Instead of always starting from the beginning of memory, next-fit starts scanning from the location where the last allocation was made. It then places the process in the next available block that is large enough. This strategy spreads allocations more evenly across memory and avoids repeatedly filling holes near the beginning. However, it may sometimes

leave larger free blocks at the start of memory unused and, in some cases, result in longer search times compared to first-fit.

5.4 *Non-contiguous memory allocation(paging , Segmentation)*

Motivation for Non-Contiguous Memory Allocation

Efficient memory management is a fundamental responsibility of an operating system. In early memory allocation strategies, processes were required to occupy a single contiguous block of physical memory. While simple, this approach suffered from the problem of *external fragmentation*, where sufficient memory may exist in total but is scattered in small fragments, making it impossible to allocate to larger processes.

To overcome this limitation, the concept of *non-contiguous memory allocation* was introduced. In this approach, the physical address space of a process does not need to be stored in a single block; instead, it can be divided into chunks located at different places in memory. This increases flexibility and allows better utilization of memory.

Two primary schemes used to achieve this are **paging** and **segmentation**. Paging divides memory into fixed-size blocks, while segmentation divides memory into variable-sized segments corresponding to program structures. These two techniques may also be combined into a hybrid scheme, providing the benefits of both. Importantly, non-contiguous memory allocation requires hardware support in the form of a Memory Management Unit (MMU), which translates logical addresses to physical addresses in real time.

1. *Paging: Mechanism and Working Principle*

Paging is one of the most widely adopted methods for non-contiguous memory allocation. The central idea is to divide both logical and physical memory into fixed-size blocks. Physical memory is partitioned into *frames*, while logical memory is divided into equally sized *pages*.

When a process consisting of n pages is loaded into memory, the operating system finds n free frames and places each page into one of these frames. Since pages do not need to be stored in consecutive frames, paging eliminates the problem of external fragmentation.

To maintain consistency between logical and physical memory, the operating system relies on a *page table*, which is created for each process. The page table maps the page number (from the logical address) to the corresponding frame number (in physical memory). When the CPU generates a logical address, the MMU uses this mapping to produce the correct physical address, allowing processes to use a continuous logical address space despite being scattered in physical memory.

Advanced Paging: CPU and Kernel Support

Although the fundamental principle of paging is consistent, modern CPUs and operating systems provide advanced features to improve flexibility and performance. Typically, page sizes range from 512 bytes to 8 KB, depending on the architecture. However, some systems support much larger page sizes, such as 16 MB, which are useful for memory-intensive applications.

Advanced operating systems like *Solaris* provide support for multiple page sizes simultaneously, for example 8 KB and 4 MB. This allows smaller pages to be used for lightweight processes, minimizing internal fragmentation, while larger pages can be used for bigger processes, reducing page table overhead.

Some systems go further by allowing *variable page sizes*, adapting dynamically at runtime to match workload requirements. This dynamic adjustment ensures efficient memory management across diverse applications, from small programs to large-scale databases.

Address Translation Scheme

Modern computer systems utilize **virtual memory** to abstract physical memory, allowing programs to operate as if they have access to a large, contiguous block of memory. The **Address Translation Scheme** is the mechanism by which the CPU translates a *virtual address* (used by programs) into a *physical address* (used by memory hardware).

Components of a Virtual Address

A virtual address generated by the CPU is divided into two key parts:

- **Page Number (p):**

- Acts as an index into the *page table*, which stores the base addresses of pages in physical memory.
- Helps the memory management unit (MMU) locate the correct page frame in physical memory.
- Uses $m - n$ bits, where:

- * m is the total number of bits in the virtual address.
- * n is the number of bits used for the page offset.

- **Page Offset (d):**

- Specifies the exact location within the page.
- Once the base address of the page is found via the page number, the offset is added to locate the specific byte or word.
- Uses n bits, which corresponds to a *page size* of 2^n bytes.

Address Structure

If the logical address space is 2^m and the page size is 2^n , then:

- The virtual address is m bits long.
- The upper $m - n$ bits represent the **page number**.
- The lower n bits represent the **offset** within the page.

This division enables the system to efficiently map virtual addresses to physical memory locations using a **page table**, which is a critical data structure in memory management.

Paging Model of Logical and Physical Memory Figure (5.6) illustrates the paging model used to translate logical memory addresses into physical memory locations. In this model, logical memory is divided into fixed-size pages (e.g., page 0, page 1, etc.), which are mapped to physical memory frames through a *page table*. Each entry in the page table corresponds to a page number and stores the frame number where that page resides in physical

memory. For example, logical page 0 is mapped to frame 1, page 1 to frame 4, and so on. This indirection allows the operating system to manage memory more flexibly, enabling non-contiguous allocation of physical memory and supporting features such as virtual memory, protection, and efficient memory utilization.

Paging Model of Logical and Physical Memory

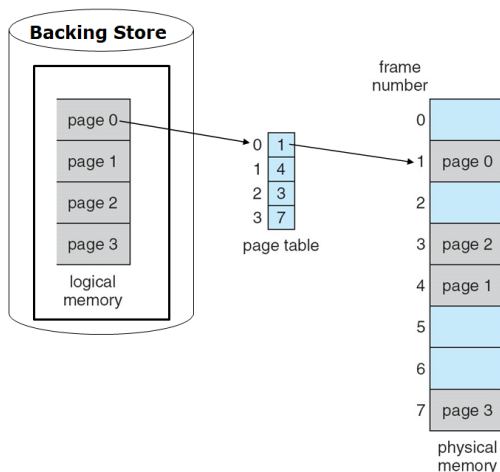


Figure 5.6: *Logical and physical address example*

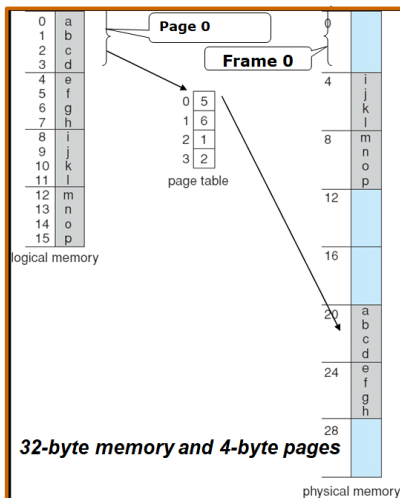
Paging Example

Logical address 0:
 page 0 offset 0.
Mapped to physical address:
 20 ($5 \times 4 + 0$)

Logical address 3:
 page 0 offset 3.
Mapped to physical address:
 23 ($5 \times 4 + 3$)

Logical address 4:
 page 1 offset 0.
Mapped to physical address:
 24 ($6 \times 4 + 0$)

Logical address 13:
 page 3 offset 1.
Mapped to physical address:
 9 ($2 \times 4 + 1$)



Operating Systems, CSE Dept., Qatar University

Figure 5.7: Paging example

Paging Example This example demonstrates how logical addresses are translated into physical addresses using a page table in a system with 32 bytes of memory and 4-byte pages. Since each page is 4 bytes, the offset within a page is determined by the last 2 bits of the logical address (because $2^2 = 4$), and the remaining bits represent the page number.

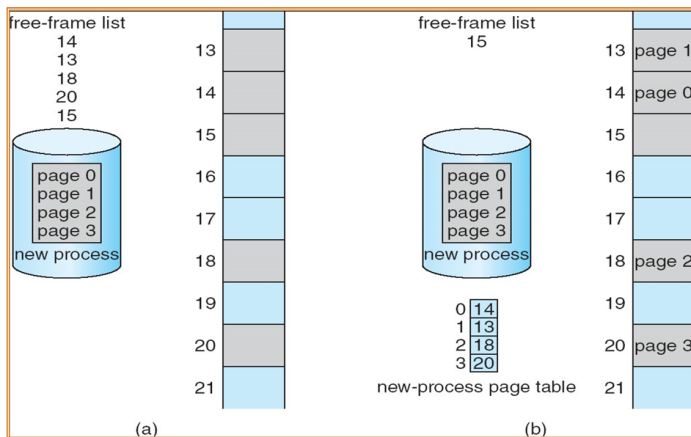
- **Logical address 0:** Page 0, Offset 0. Page 0 maps to frame 5.
 Physical address = $5 \times 4 + 0 = 20$.

- **Logical address 3:** Page 0, Offset 3. Page 0 maps to frame 5.
Physical address = $5 \times 4 + 3 = 23$.
- **Logical address 4:** Page 1, Offset 0. Page 1 maps to frame 6.
Physical address = $6 \times 4 + 0 = 24$.
- **Logical address 13:** Page 3, Offset 1. Page 3 maps to frame 11.
Physical address = $11 \times 4 + 1 = 45$.

This example clearly shows how logical addresses are divided into a page number and an offset, and how the page table is used to find the corresponding frame in physical memory. The final physical address is calculated by multiplying the frame number by the page size and adding the offset.

Free Frame Allocation in Paging Systems

Free Frames



Operating Systems, CSE Dept., Qatar University

Figure 5.8: *Frames example*

The diagram illustrates the allocation of free frames to a new process in a paged memory system. In the *Before allocation* state, the free-frame list contains available physical memory frames such as 14, 18, 20, and 15. When a new process is created, it requires several pages to be loaded into memory. In the *After allocation* state, the operating system assigns free frames to the process's pages: page 0 is placed in frame 14, page 1 in frame 18, page 2 in

frame 3, and page 3 in frame 20. These mappings are recorded in the new process's page table, enabling the system to translate logical page numbers into physical frame numbers. This example demonstrates the dynamic nature of memory allocation in paging systems, where frames are drawn from a pool of free frames and updated in the page table to support efficient and flexible memory management.

Shared Pages in Memory Management

Shared Code

- A single copy of read-only (reentrant) code is shared among multiple processes.
- Common examples include text editors, compilers, and window systems.
- The shared code must reside at the same logical address in each process's address space.

Private Code and Data

- Each process maintains its own separate copy of code and data.
- These private pages are not shared and may be located anywhere in the process's logical address space.

Rules for practical Problems

Example

Consider a system with 2 GB (2^{31} bytes) of memory and a frame size of 4 KB (2^{12} bytes). A process has a logical address space of size 250 KB. The page table allocates frame number $k + 1,000$ to page k of this process. Below are the answers to several paging-related questions:

a) Physical Address of Logical Address 443322

$$\text{Virtual Logical Address} = 443322$$

$$\text{Page Number} = \left\lfloor \frac{443322}{4096} \right\rfloor = 108$$

$$\text{Offset} = 443322 \bmod 4096 = 210$$

$$\text{Frame Number} = 108 + 1000 = 1108$$

$$\text{Physical Address} = (1108 \times 4096) + 210 = 4542218$$

b) Amount of Internal Fragmentation

$$\begin{aligned} \text{Internal Fragmentation} &= \text{Page Size} - (\text{Process Size} \bmod \text{Page Size}) \\ &= 4096 - (250000 \bmod 4096) = 409 \end{aligned}$$

c) Number of Pages Needed

$$\text{Number of Pages} = \left\lceil \frac{250000}{4096} \right\rceil = \lceil 61.04 \rceil = 62$$

d) Number of Frames in the System

$$\text{Number of Frames} = \frac{2 \text{ GB}}{4 \text{ KB}} = \frac{2 \times 2^{30}}{2^{12}} = 2^{19} = 524288$$

162
201

e) Page Table Size

- **Number of entries:** $\lceil \log_2(\text{number of pages}) \rceil = \lceil \log_2(62) \rceil = 6$
- **Frame number size:** $\lceil \log_2(4096) \rceil = 12$
- **Total page table size:** $62 \times (6 + 12) = 62 \times 18 = 1116$ bits

f) If an entry in the page table consists of a frame number and the offset, what is the size of the page table for this process in bits?

Solution:

$$\begin{aligned}\text{Page Table Size} &= (\# \text{Bits to represent frame} + \# \text{Bits to represent offset}) \times \text{number of pages} \\ &= (19 + 12) \times 63 \\ &= 31 \times 63 = 1953 \text{ bits}\end{aligned}$$

Discussion

This example demonstrates key concepts in paging, including address translation, fragmentation, and page table sizing. The internal fragmentation arises because the last page may not be fully utilized, leading to wasted space. The page table must store both the page number and corresponding frame number, and its size depends on the number of pages and the number of bits required to represent each. Efficient memory management requires balancing fragmentation, page table overhead, and address translation speed.

Practice Consider a system with a frame size of 4 KB. If a process P has a logical address space of size 200200 bytes:

- a) **How many pages are needed to represent this process?**

$$\text{Pages} = \left\lceil \frac{200200}{4096} \right\rceil = \lceil 48.876953125 \rceil = 49$$

Frames numbered: 0 to 48

- b) **What is the amount of internal fragmentation, if any?**

$$\begin{aligned} \text{Internal Fragmentation} &= 4096 - (200200 \bmod 4096) \\ &= 4096 - 3592 = 504 \text{ bytes} \end{aligned}$$

- c) **Assume that page k of this process is assigned to frame $k + 1000$. Find the virtual and physical address for each of the following logical addresses of process P :**

- **66690:**

$$\text{Page number} = \left\lfloor \frac{66690}{4096} \right\rfloor = 16$$

$$\text{Offset} = 66690 \bmod 4096 = 1154$$

$$\text{Frame number} = 16 + 1000 = 1016$$

$$\text{Physical address} = (1016, 1154)$$

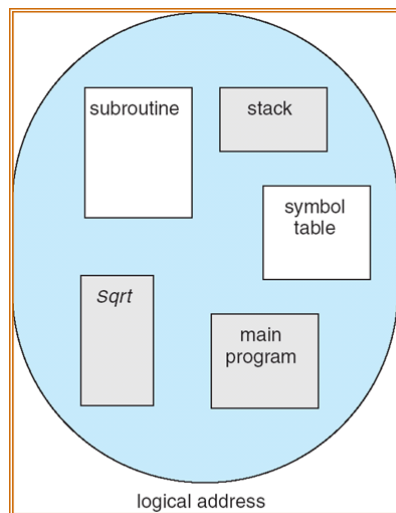
- **555011:** similar to (i)

2. *Segmentation*

Overview

Segmentation is a memory-management scheme that supports the user's view of memory. Unlike paging, which divides memory into fixed-size

blocks, segmentation divides memory into variable-sized segments that correspond to logical units of a program. This approach aligns more closely with how programmers and compilers structure programs.



User's view of the program

Concept

A program is typically composed of multiple segments, each representing a distinct logical unit. These segments may include:

- **Main program**
- **Procedure**

- **Function**
- **Method**
- **Object**
- **Local variables and global variables**
- **Common blocks**
- **Stack**
- **Symbol tables and arrays**

Each segment is defined by its base address and length. The base address specifies the starting physical location of the segment in memory, while the length defines its size. The logical address generated by the CPU is a pair: (segment number, offset). The offset must be less than the segment length; otherwise, a segmentation fault occurs.

User's View of Memory

Segmentation provides a user-centric view of memory, allowing programs to be divided into meaningful components. This abstraction simplifies programming and enhances modularity. For example, a user may view memory as consisting of separate regions for the main program, stack, subroutines, and symbol tables.

Advantages

- Facilitates modular program design and compilation.
- Enables protection and sharing at the segment level.

- Supports dynamic memory allocation with variable-sized segments.
- Enhances logical organization and debugging.

Challenges

- External fragmentation may occur due to variable segment sizes.
- Requires

Segmentation Architecture

In segmentation-based memory management, a **logical address** is represented as a tuple:

$$\langle \text{segment number, offset} \rangle$$

This format enables memory access based on logical segments rather than a linear address space.

Segment Table

The **segment table** is a data structure that maps each segment to its corresponding physical memory location. Each entry in the segment table contains:

- **Base:** The starting physical address of the segment in memory.
- **Limit:** The length of the segment, defining its size.

These values are used to validate and translate logical addresses into physical addresses.

Segment Table Registers

To facilitate segmentation, the system uses two special registers:

- **Segment-Table Base Register (STBR):** Stores the starting address of the segment table in memory.
- **Segment-Table Length Register (STLR):** Indicates the total number of segments a program is permitted to use.

Address Validation

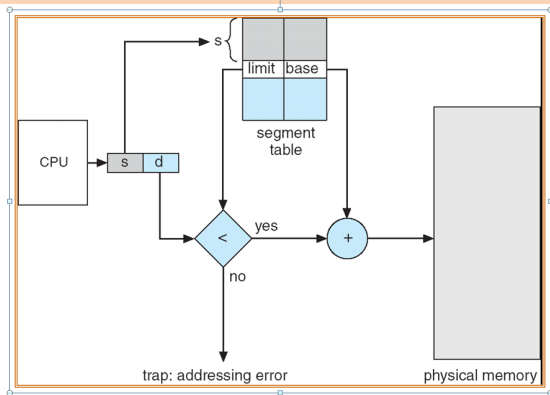
Before accessing a segment, the system verifies that the segment number is within bounds. A segment number s is valid only if:

$$s < \text{STLR}$$

If this condition is not satisfied, the reference is considered illegal and may trigger a segmentation fault. memory management and segment tables.

- Segment relocation and protection checks add overhead.

Segmentation Hardware



Practice 1

Consider the following segmentation table for the process P that has zero internal fragmentation:

Segment	Base	Limit
0	11140000	5500
1	11110000	2600
2	10010000	1400
3	16170000	3800

- Find the physical address of virtual logical address (1, 2300).

Answer:

Segment 1 base = 11110000

Offset = 2300

Physical address = $11110000 + 2300 = 11112300$

2. What is the logical address of the physical address 16171234?

Answer:

Segment 3 base = 16170000

Logical address = $(3, 16171234 - 16170000) = (3, 1234)$

3. Find the physical address of the logical address 8300.

Answer:

Logical address 8300 falls in segment 2:

$8300 - 5500 = 2800$ (in segment 2)

Segment 2 base = 10010000

Physical address = $10010000 + 2800 = 10010200$

Practice 2

Consider the following segmentation table for the process P that has zero internal fragmentation:

Segment	Base	Limit
0	11100000	8600
1	11140000	3000
2	10010000	2400
3	16170000	500

1. What is the total size of the logical address space?

$$\text{Total size} = 8600 + 3000 + 2400 + 500 = \boxed{14500}$$

2. Find the physical address of virtual logical address (2, 450).

$$\text{Segment 2 base} = 10010000$$

$$\text{Offset} = 450$$

$$\text{Physical address} = 10010000 + 450 = \boxed{10010450}$$

3. Find the virtual logical address of the logical address 11300.

$$11300 - 8600 = 2700 \quad (\text{less than } 3000, \text{ so in segment 1})$$

$$\text{Virtual logical address} = \boxed{(1, 2700)}$$

5.5 *Exercise*

- 1. What requirements is memory management intended to satisfy?**
- 2. What is relocation of a program?**
- 3. What are the advantages of organizing programs and data into modules?**
- 4. What are some reasons to allow two or more processes to all have access to a particular region of memory?**
- 5. In a fixed partitioning scheme, what are the advantages of using unequal-size partitions?**

6. **What is the difference between internal and external fragmentation?**
7. **What is the difference between a page and a frame?**
8. **What is the difference between a page and a segment?**

Paging and Address Translation Practice

1. **What is the page number for the logical address 765432 in a system with an 8 KB page size?**
2. **If frame number 300 is assigned to page 93, what is the physical address of logical address 765432?**
3. **How many pages are required for a process of size 1.2 MB with an 8 KB page size?**
4. **How many frames are there in a system with 4 GB of memory and 8 KB frame size?**
5. **How many bits are required to represent the page number for a process with 151 pages?**
6. **What is the number of bits required to represent the offset for a page size of 16 KB?**
7. **What is the physical address for logical address 567890 if the page number is 69, offset is 567, and the frame number is 3069?**

8. A segment table has the following entry:

Segment 0: Base = 3000, Limit = 1500

What is the physical address for the logical address (0, 1200)?

9. What is the total logical address space for a system with the following segment table:

- Segment 0: Limit = 2000
- Segment 1: Limit = 3000
- Segment 2: Limit = 1500

Chapter 6

Uni Processor Scheduling

6.1 Introduction to Scheduling in Operating Systems

Scheduling is a core function of an operating system that determines how processes are selected for execution. Since system resources are limited, the OS must manage multiple processes efficiently to ensure fairness, responsiveness, and optimal performance.

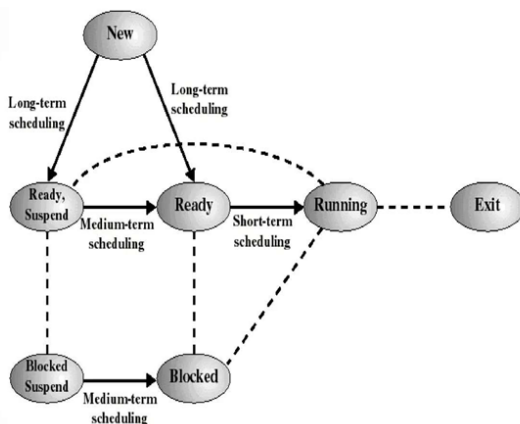
Goals of Scheduling

- Maximize CPU utilization
- Minimize waiting time, turnaround time, and response time
- Ensure fairness among processes
- Support different types of workloads (interactive, batch, real-time)

Types of Scheduling

Scheduling is typically divided into three categories:

Type	Role in Process Management	Frequency
Long-term	Decides which processes to admit into the system (from New to Ready)	Infrequent
Medium-term	Manages swapping of processes in/out of memory (between Ready/Blocked and Suspended)	Occasional
Short-term	Chooses which ready process to execute next (from Ready to Running)	Frequent



This diagram illustrates the lifecycle of a process and the role of different schedulers.

Key Transitions

- **New → Ready (Long-term Scheduling):** The long-term scheduler admits new processes into the ready queue, controlling the degree of multiprogramming.
- **Ready → Running (Short-term Scheduling):** The short-term scheduler selects a process from the ready queue to execute.
- **Running → Exit:** The process completes and terminates.
- **Running → Ready:** If preempted, the process returns to the ready queue.
- **Running → Blocked:** The process waits for I/O or a resource.
- **Blocked → Ready:** Once I/O completes, the process returns to the ready queue.
- **Ready/Blocked ↔ Suspended (Medium-term Scheduling):** The medium-term scheduler swaps processes in and out of memory to manage resource usage.

6.2 types of scheduling

Long-Term Scheduling

Long-term scheduling is primarily concerned with determining which programs are admitted into the system for processing. Its role is crucial because it sets the foundation for system performance by controlling the degree of multiprogramming. The scheduler decides how many and

which processes should be loaded into the ready queue, balancing system load and efficiency.

If more processes are admitted into memory, there are several consequences:

- The likelihood of all processes being blocked simultaneously is reduced, thereby maintaining system responsiveness.
- Better CPU utilization is achieved since there are more processes available to occupy the processor when others are waiting for I/O operations.
- With more processes, each one receives a smaller fraction of CPU time, leading to longer waiting times for completion.

Thus, the long-term scheduler seeks to maintain an equilibrium by sometimes attempting to keep a mix of CPU-bound and I/O-bound processes, ensuring efficient use of resources while minimizing bottlenecks.

Medium-Term Scheduling

Medium-term scheduling is concerned with swapping decisions, which are necessary for managing the degree of multiprogramming in the system. Its function is to temporarily remove (or suspend) certain processes from main memory and later reintroduce them, depending on system conditions. This approach allows the long-term scheduler to admit more processes than the memory can simultaneously accommodate.

However, this introduces trade-offs: while admitting more processes increases concurrency, it can also lead to excessive paging or disk activity if too many processes compete for memory, degrading performance.

Therefore, medium-term scheduling tries to maintain an “optimum” level of multiprogramming, avoiding overloading the system.

It is usually implemented as part of memory management software, as memory allocation, swapping, and paging decisions are central to its operation. By dynamically adjusting the set of processes in memory, medium-term scheduling improves system performance and responsiveness under fluctuating workloads.

Short-Term Scheduling

Short-term scheduling, also known as CPU scheduling, determines which process among the ready queue will execute next. Unlike long-term and medium-term scheduling, it operates frequently and makes rapid decisions. Its focus is on immediate CPU allocation, ensuring efficient and fair utilization of processor time.

This scheduler is invoked by various system events, such as:

- Clock interrupts (indicating the end of a time slice).
- I/O interrupts (when a process completes or requests I/O).
- Operating system calls and traps.
- External signals.

Each of these events provides an opportunity to reassess which process should execute next. The short-term scheduler plays a direct role in maintaining system performance, as its policies—like First-Come-First-Served (FCFS), Shortest Job Next (SJN), Round Robin (RR), or Priority Scheduling—determine process responsiveness, turnaround time, and

CPU efficiency. Thus, it is the most active of the three schedulers and has a direct impact on user experience.

6.3 Understanding the CPU-I/O Cycle

Processes in operating systems alternate between CPU execution and I/O operations, forming the CPU-I/O cycle. Based on their behavior, processes are classified as either CPU-bound or I/O-bound. This classification is crucial for optimizing scheduling and resource allocation.

CPU-bound Processes

CPU-bound processes primarily utilize the CPU for extended periods, performing intensive computations with minimal I/O interaction. These processes benefit from faster CPUs and parallel execution strategies.

I/O-bound Processes

I/O-bound processes frequently perform input/output operations and spend most of their time waiting for I/O completion. Their performance depends on the efficiency of I/O devices and subsystems.

Aspect	CPU-bound	I/O-bound
Primary Activity	Computation-intensive	I/O-intensive
CPU Burst Duration	Long	Short
I/O Wait Duration	Short	Long
Performance Bottleneck	CPU	I/O subsystem
Optimization Strategy	CPU scheduling, parallelism	I/O throughput, latency reduction
Typical Applications	Simulations, data analysis	Web servers, file access

Table 6.1: *Comparison between CPU-bound and I/O-bound processes*

6.4 Scheduling Criteria

Process scheduling is a fundamental aspect of operating system design, directly influencing both system performance and user satisfaction. To evaluate the effectiveness of scheduling algorithms, various criteria are used—some focusing on the user’s experience and others on the system’s overall efficiency. These criteria help determine how well an operating system manages its resources and responds to user demands. Broadly, they can be categorized into **user-oriented** and **system-oriented** metrics.

User-Oriented Scheduling Criteria

User-oriented criteria emphasize the responsiveness and completion time of processes from the user's perspective. These are particularly important in environments where user interaction or job completion time is critical.

- **Turnaround Time (Batch Systems)**

Turnaround time refers to the total elapsed time from the submission of a process to its completion. It includes all phases of execution—waiting, processing, and I/O. In batch systems, where users submit jobs and wait for results, minimizing turnaround time is essential for user satisfaction.

- **Response Time (Interactive Systems)**

Response time measures the interval between submitting a request and receiving the first response. This metric is crucial in interactive systems such as graphical user interfaces or networked applications, where users expect immediate feedback. Lower response times enhance the perceived responsiveness of the system.

System-Oriented Scheduling Criteria

System-oriented criteria focus on optimizing resource usage and maintaining fairness across processes. These metrics are vital for ensuring that the system operates efficiently and equitably.

- **CPU Utilization**

CPU utilization indicates the percentage of time the CPU is actively executing processes. High CPU utilization suggests efficient use of

processor resources, while low utilization may indicate idle time and poor scheduling.

- **Fairness**

Fairness ensures that all processes receive a reasonable share of CPU time, preventing any single process from monopolizing resources. A fair scheduler avoids starvation and promotes equitable access to system resources.

- **Throughput**

Throughput is defined as the number of processes completed per unit time. It reflects the system's productivity and is a key metric in environments where maximizing completed tasks is a priority.

Scheduling Policies

Scheduling policies determine how processes are selected and managed for execution on the CPU. These policies are essential for ensuring efficient resource utilization, maintaining system responsiveness, and enforcing fairness among processes. Scheduling policies are typically defined by two key components: the **selection function** and the **decision mode**. Each plays a distinct role in shaping the behavior of the scheduler.

Selection Function

The selection function specifies *which process in the ready queue* should be chosen next for execution. This decision is based on the scheduling algorithm in use, which may consider factors such as priority, arrival time, or expected execution duration. The goal of the selection function

is to optimize performance metrics like turnaround time, response time, or CPU utilization.

Decision Mode

The decision mode defines *when* the selection function is applied—that is, at what points in time the operating system decides to switch processes. There are two primary modes:

- **Nonpreemptive Scheduling**

In this mode, once a process is selected and begins execution, it continues running until it either completes or voluntarily yields the CPU (e.g., by blocking for I/O). This approach simplifies scheduling but may lead to poor responsiveness, especially if long processes monopolize the CPU.

- **Preemptive Scheduling**

The operating system can interrupt a currently running process and move it back to the ready queue, allowing another process to execute. This mode improves responsiveness and fairness by preventing any single process from monopolizing the CPU. It is essential in time-sharing and real-time systems where quick reaction to external events is required.

6.5 Scheduling Algorithms

1-First Come First Served (FCFS) Scheduling

First Come First Served (FCFS) is one of the simplest scheduling algorithms used in operating systems. It operates on the principle of a **FIFO**

(First-In-First-Out) queue, where the process that arrives first is executed first. This method is intuitive and easy to implement but comes with notable limitations in terms of responsiveness and efficiency.

Mechanism

- **Selection Function:** The scheduler selects the process that has been waiting the longest in the ready queue. This is purely based on arrival time, without considering process length or priority.
- **Decision Mode:** FCFS is a **nonpreemptive** algorithm. Once a process starts execution, it runs until it either completes or blocks itself (e.g., for I/O). The CPU is not taken away from the running process until it voluntarily releases it.

Example

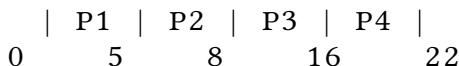
Consider the following set of processes with their arrival and burst times:

Process	Arrival Time	Burst Time
P ₁	0	5
P ₂	1	3
P ₃	2	8
P ₄	3	6

Execution Order

Based on arrival time: $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$

Gantt Chart Representation



Turnaround and Waiting Times

Turnaround Time (TAT) = Completion Time – Arrival Time

Waiting Time (WT) = Turnaround Time – Burst Time

Process	Completion Time	TAT	WT
P ₁	5	5	0
P ₂	8	7	4
P ₃	16	14	6
P ₄	22	19	13

Drawbacks

- **Convoy Effect:** Short processes may get stuck behind long ones, leading to poor average waiting time and reduced responsiveness.
- **Poor for Interactive Systems:** Since it is non preemptive, FCFS is unsuitable for systems requiring quick response times (e.g., GUIs, real-time applications).
- **No Prioritization:** All processes are treated equally regardless of urgency or resource needs, which can lead to inefficient CPU usage.

- **High Average Waiting Time:** Especially when long processes arrive early, they delay all subsequent processes.

2-Shortest Job First (SJF) Scheduling

Shortest Job First (SJF) is a scheduling algorithm that selects the process with the **shortest expected CPU burst time** for execution. It aims to minimize average waiting time and turnaround time by prioritizing shorter tasks. SJF is optimal in terms of these metrics under certain conditions, but it also has practical limitations.

Mechanism

- **Selection Function:** The scheduler chooses the process with the shortest estimated CPU burst time from the ready queue. This favors I/O-bound or short-duration processes.
- **Decision Mode:** SJF is typically implemented as a **non-preemptive** algorithm. Once a process starts executing, it runs to completion without interruption.
- **Burst Time Estimation:** Since future burst times are not known in advance, SJF relies on *past behavior* to estimate the expected CPU time for each process. This can be done using techniques like exponential averaging.

Example

Consider the following processes with their arrival and burst times:

Process	Arrival Time	Burst Time
P ₁	0	8
P ₂	1	4
P ₃	2	2
P ₄	3	1

Execution Order

Based on shortest burst time: P₁ → P₄ → P₃ → P₂

Gantt Chart Representation

	P ₁	P ₄	P ₃	P ₂	
0	8	9	11	15	

Turnaround and Waiting Times

Turnaround Time (TAT) = Completion Time – Arrival Time

Waiting Time (WT) = Turnaround Time – Burst Time

Process	Completion Time	TAT	WT
P ₁	8	8	0
P ₄	9	6	5
P ₃	11	9	7
P ₂	15	14	10

Drawbacks

- **Requires Accurate Burst Time Estimation:** Inaccurate predictions can lead to suboptimal scheduling decisions.
- **Starvation of Long Processes:** Longer processes may be delayed indefinitely if shorter ones keep arriving.
- **Not Suitable for Interactive Systems:** Being non-preemptive, SJF cannot respond quickly to user inputs or time-sensitive tasks.
- **Complexity in Estimation:** Estimating burst time based on past behavior adds overhead and may not always reflect future execution accurately.

3-Shortest Remaining Time (SRT) Scheduling

Shortest Remaining Time (SRT) is the **preemptive version of Shortest Job First (SJF)** scheduling. It selects the process with the **least remaining CPU burst time** and allows preemption if a new process arrives with a shorter burst time than the currently running one.

Mechanism

- The scheduler continuously monitors the ready queue.
- If a new process arrives with a burst time less than the remaining time of the current process, the CPU **preempts** the current process.
- This ensures that the process with the **shortest remaining time** always runs.

Example

Process	Arrival Time	Burst Time
P ₁	0	8
P ₂	1	4
P ₃	2	2

Non-Preemptive SJF Execution

- P₁ runs first (no preemption), then P₃, then P₂.
- Completion Times: P₁ = 8, P₃ = 10, P₂ = 14
- Turnaround Times: P₁ = 8, P₃ = 8, P₂ = 13
- **Average Turnaround Time** = $(8 + 8 + 13)/3 = 9.67$

Preemptive SJF (SRT) Execution

- P₁ starts, but is preempted by P₂ at time 1.
- P₂ is then preempted by P₃ at time 2.
- Completion Times: P₃ = 4, P₂ = 8, P₁ = 16
- Turnaround Times: P₃ = 2, P₂ = 7, P₁ = 16
- **Average Turnaround Time** = $(2 + 7 + 16)/3 = 8.33$

Advantages

- **Improved Turnaround Time:** Short jobs finish quickly, reducing average turnaround time.
- **Responsive to New Arrivals:** New short jobs can preempt long ones immediately.
- **Reduced Waiting Time:** Short processes spend less time in the ready queue.
- **Better CPU Utilization:** CPU always works on the most efficient task.
- **Prevents Long Job Domination:** Long jobs can't monopolize the CPU.
- **Suitable for Interactive Systems:** Ideal for environments needing fast response times.

Disadvantages

- **High Complexity:** Requires constant tracking of remaining burst times.
- **Inaccurate Burst Time Estimation:** Poor estimates lead to inefficient scheduling.
- **Frequent Context Switching:** Preemption increases overhead due to switching.
- **Starvation of Long Jobs:** Long jobs may be delayed indefinitely.

- **Unfair to Long Jobs:** Short jobs are favored, long ones suffer.
- **Not Suitable for Real-Time Systems:** Lack of predictability makes it unreliable for time-critical tasks.

Comparison Table

Feature	Preemptive (SRT)	SJF	Non-Preemptive SJF
Decision Timing	Continuously checks for new arrivals		Only at process completion
Preemption	Yes		No
Responsiveness to Short Jobs	High – immediate CPU access		Lower – must wait for current job
Complexity	Higher – needs tracking of remaining time		Simpler
Turnaround Time	Often better for short jobs	for	May be worse if long job starts first

4-Round-Robin (RR) Scheduling

Round-Robin (RR) is a **preemptive scheduling algorithm** designed for **time-sharing systems**. Each process is assigned a fixed time slice (quantum), and the CPU cycles through the ready queue in a **First-Come, First-Served (FCFS)** manner.

Mechanism

- The scheduler maintains a ready queue of processes.
- Each process receives a fixed time slice (e.g., 10–100 ms).
- If a process does not finish within its time slice, it is preempted and placed at the end of the queue.
- The CPU continues cycling through the queue until all processes complete.

Key Characteristics

Feature	Description
Selection Function	Same as FCFS — picks the first process in the queue
Decision Mode	Preemptive — enforced by a timer interrupt
Time Slice	Fixed duration (e.g., 10–100 ms), determines responsiveness and overhead
Queue Behavior	After preemption, process goes to the tail of the ready queue

Example

Assume:

- Time quantum = 4 ms

- Processes: P₁ (burst = 10), P₂ (burst = 4), P₃ (burst = 6)

Execution Order

- P₁ runs for 4 ms → remaining = 6
- P₂ runs for 4 ms → completes
- P₃ runs for 4 ms → remaining = 2
- P₁ resumes for 4 ms → remaining = 2
- P₃ resumes for 2 ms → completes
- P₁ resumes for 2 ms → completes

Completion Times

- P₂ = 8, P₃ = 14, P₁ = 16
- Turnaround Times: P₂ = 8, P₃ = 12, P₁ = 16
- Average Turnaround Time** = $(8 + 12 + 16)/3 = 12$

Advantages

- Fairness:** Every process gets equal CPU time in turn.
- Preemptive by Design:** Prevents CPU monopolization by long processes.
- Good for Time-Sharing:** Ideal for interactive systems and multi-user environments.
- Simple Implementation:** Easy to understand and implement with a circular queue.

Disadvantages

- **High Context Switching Overhead:** Frequent preemption increases CPU overhead.
- **Poor for Short Jobs:** Short jobs may wait behind long ones in the queue.
- **Quantum Size Sensitivity:**
 - Too small → excessive context switching
 - Too large → behaves like FCFS
- **No Priority Handling:** All processes treated equally, regardless of urgency.

Comparison with Other Algorithms

Feature	Round-Robin (RR)	FCFS	SJF / SRT
Preemption	Yes	No	SJF: No, SRT: Yes
Fairness	High	Low	Low (short jobs favored)
Turnaround Time	Moderate	Can be poor	Often best for short jobs
Complexity	Low	Very low	SRT: High
Context Switching	High	None	SRT: Moderate to High
Real-Time Suitability	Limited	Poor	SRT: Poor

6.6 Exercise

Multiple Choice Questions

1. Which of the following scheduling algorithms is non-preemptive?

(a) Round Robin

- (b) Shortest Job First (SJF)
 - (c) Priority Scheduling (Preemptive)
 - (d) Multilevel Queue
2. **Which scheduling algorithm minimizes the average waiting time for a fixed set of processes?**
- (a) Round Robin
 - (b) FCFS
 - (c) SJF
 - (d) Priority
3. **What is the key drawback of Round Robin scheduling when the time quantum is too small?**
- (a) Increased waiting time
 - (b) Starvation of processes
 - (c) High context-switching overhead
 - (d) Poor throughput
4. **What happens if the time quantum in Round Robin scheduling is too large?**
- (a) It behaves like FCFS
 - (b) Starvation occurs
 - (c) It leads to high overhead

- (d) Processes are executed out of order
5. Which scheduling algorithm selects the process with the shortest burst time from the ready queue without preemption?
- a) Round-Robin (RR)
 - b) Shortest Job First (SJF)
 - c) Shortest Remaining Time (SRT)
 - d) First-Come, First-Served (FCFS)
6. In Round-Robin scheduling, what determines how long a process runs before being preempted?
- a) Priority level
 - b) Burst time
 - c) Time quantum
 - d) Arrival time
7. Which of the following is a disadvantage of First-Come, First-Served (FCFS) scheduling?
- a) High context switching overhead
 - b) Starvation of short jobs
 - c) Complex implementation
 - d) Requires burst time estimation
8. Which algorithm is most suitable for interactive systems requiring quick response times?

- a) Shortest Job First (SJF)
 - b) First-Come, First-Served (FCFS)
 - c) Round-Robin (RR)
 - d) Shortest Remaining Time (SRT)
9. What is the main reason SRT can cause starvation?
- a) Processes are executed randomly
 - b) Long jobs are continuously preempted by short ones
 - c) It uses a large time quantum
 - d) It does not consider arrival time
10. Which algorithm is the simplest to implement among the following?
- a) Shortest Remaining Time (SRT)
 - b) Round-Robin (RR)
 - c) Shortest Job First (SJF)
 - d) First-Come, First-Served (FCFS)
11. What happens in Round-Robin scheduling if the time quantum is set too small?
- a) Processes finish faster
 - b) Context switching overhead increases
 - c) It behaves like SJF
 - d) Long jobs are prioritized

6.7 Solve the Problems

- Consider the following processes with arrival times and burst times:

Process	Arrival Time	Burst Time
P ₁	0	5
P ₂	1	3
P ₃	2	8

Using **First-Come, First-Served (FCFS)** scheduling, compute the average waiting time.

- Consider a round Robbin problem

Process	Arrival Time (ms)	Burst Time (ms)
P ₁	0	9
P ₂	1	5
P ₃	2	2
P ₄	3	7

Time Quantum: 3 ms

Tasks

1. Compute Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

2. Find the Average Turnaround Time and Average Waiting Time.

Bibliography

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- [2] Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems* (4th ed.). Pearson.
- [3] Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
- [4] Bach, M. J. (1986). *The Design of the UNIX Operating System*. Prentice Hall.
- [5] Love, R. (2010). *Linux Kernel Development* (3rd ed.). Addison-Wesley.
- [6] Milankovic, M. (1992). *Operating Systems: Concepts and Design*. McGraw-Hill.